

Important

本笔记建议搭配 *Typora* 食用以获得最佳体验，或者直接进入PDF查看

C++练习笔记

- 0 点子王
- 1 printf 和 scanf 语法初识
- 2 取整函数
- 3 位运算(简单版)
- 4 sort()排序函数
- 5 __gcd求最大公约数函数
- 6 字符与字符串
- 7 vector 容器 / 动态数组
- 8 栈(stack)
- 9 宏常量定义小寄巧
- 10 数组前导零与后导零的删除
- 11 位运算抽象版
- 12 时间函数帮你计算运算时间
- 13 一组数的各种数计算
 - 平均数:
 - 中位数:

算法

算法大纲(登神长阶)

- 1 顺序表
- 2 链表
- 3 栈
- 4 队列
- 5 字符串
- 6 树
- 7 图
- 8 动态规划
- 1 递归
- 2 递推
- 3 贪心
 - 分数背包问题 (Fractional Knapsack Problem)**
- 4 桶排序
- 5 高精度
 - 5.1 高精度加法
 - 5.2 高精度减法
 - 5.3 高精度乘法
- 6 STL
 - 6.1 容器
 - 6.1.1 vector
 - 6.1.1.1 尾接与尾删
 - 6.1.1.2 size函数
 - 6.1.1.3清空数组
 - 6.1.1.4 empty函数
 - [!!!] 6.1.1.5 resize函数
 - 6.1.1.6 访问vector的数据
 - 6.1.1.7 vector的赋值与读取
 - 6.1.1.8 vector的使用情况
 - 6.1.1.9 注意事项
 - 6.1.1.10 和其他容器的组合技
 - 6.1.2 栈 **stack**

- 6.1.2.1 构造方式
 - 6.1.2.2 进栈与出栈及取栈顶部
 - 6.1.1.3 用vector模拟stack
 - 6.1.1.4 写栈的注意事项
 - 6.1.2.5 与 *vector* 相比 *stack*的优势是什么?
- 6.1.3 队列 queue
 - 6.1.3.1 常用方法
 - 6.1.3.2 注意事项
- 6.1.4 优先队列(堆) priority_queue
 - 6.1.4.1 构造
 - 6.1.4.2 常用语法
 - 适用场景
 - 6.1.4.3 注意事项
- 6.1.5 集合set
 - 6.1.5.1 常用操作
 - 6.1.5.2 遍历
 - 6.1.5.3 适用范围
 - 6.1.5.4 注意事项
- 6.1.6 映射 map
 - 6.1.6.1 构造及常用操作
 - 6.1.6.2 遍历
 - 6.1.6.3 适用范围
 - 6.1.6.4 注意事项
- 6.1.7 字符串string
 - 6.1.7.1 常用方法
 - 6.1.7.2 string 转化
 - 6.1.7.3 注意事项
- 6.1.8 二元组 pair
 - 6.1.8.1 常用方法
 - 6.1.8.2 适用范围
- 6.1.9 列表list
 - 6.1.9.1 优势与适用范围
 - 6.1.9.2 常用方法
 - 6.1.9.3 list的遍历及迭代器的操作
 - 6.1.9.4 list容器的常见访问操作
 - 6.1.9.5 list 容器的常见修改操作
 - 6.1.9.6 list容器常用的操作
 - deque 双向队列
- 6.2 迭代器(遍历器)
 - 常用容器的迭代器
 - 迭代器的辅助函数
- 6.3 STL函数
 - 对容器操作类
 - 对容器改动类
- 7 快速幂
- 8 dfs 深度优先搜索
 - 使用DFS实现全排列
 - [板子] 一组数据取任意个数据进行操作
 - 方法一: DFS爆搜
 - 法二: DP
 - 例二 法一: DFS
 - 例二 法二: DP
- 9 素数筛
 - 9.1 一般双重筛
 - 9.2 埃拉托斯特尼筛法
 - 9.3 欧拉筛法

10 DP动态规划[uncomplete]

背包DP

11 状态压缩

12 哈希表

13 前缀和与差分

13.1 一维数组的前缀和

13.2 二维数组的前缀和

树上前缀和

差分

14 BFS 广度优先搜索

15 单调栈

16 约瑟夫问题

17 最长回文子序列

17.1 模拟暴力法

17.2 中心扩展算法

17.3 Manacher's Algorithm

注释

C++练习笔记

0 点子王

- 万能头文件 `#include<bits/stdc++.h>` (可以解决大部分头文件问题)
- 使用 `printf("%.2f", [输出])` 来输出保留两位的小数
- 小写字母的ASCII码是大写字母的ASCII + 32
- 在处理时间问题时，我们不妨全部转化为最小的时间单位，再用循环回复时间格式
- 字符串/字符 进行比较的时候要用`strcmp(a,b)`
 - 当 `str1 < str2` 时，返回为负数(-1);
 - 当 `str1 == str2` 时，返回值= 0;
 - 当 `str1 > str2` 时，返回正数(1)。
-  **Tip**
更新：对字符串来说也可以用 `==` 符号判等 ——> 甚至可以扩展到vector上
- 使用`stoi()`函数将字符串强转为整型
- `int i = char(数字) - '0'` 即为字符型数字的整型
- `char i = int(整型) + '0'` 即为整型数字的字符型
- 在解题的时候切忌只用算法硬算，理解数学思路并简化题目 **非常重要**
- 使用 `getline()` 函数获取长字符串 `getline(cin, str)`
- 主要当 `cin>>` 和 `getline()` 一起使用的时候要使用如下代码删除getline缓存：

```
1  int n = 0;
2      cin >> n;
3      string str;
4      getline(cin, str);
5      str = "\n";
6      getline(cin, str);
```

- 比较器函数初识
- `less<类型>` 表示升序排列[由小到大] ¹
- `greater<类型>` 表示降序排列[由大到小]
- **string**也是可以比大小的，会按照字典序来比较大小
- 关闭同步流，获得最速输入输出 **！关闭同步流后cin和cout不能和printf和scanf混用**
- `ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);` [取消同步流]
- `#pragma GCC optimize(2)` 在文件头添加这个实现O2优化
- 判断一个除法除出来的数是不是整数推荐使用取模法 `if((a*c)%b == 0) cout << "is intnum"` (血泪教训见下)

73126519	小歪商店故事: 卷	答案错误	20	9	536	601	C++	2024-11-10 20:11:52
73124099	小歪商店故事: 卷	答案错误	20	15	632	508	C++	2024-11-10 19:59:37
73122967	小歪商店故事: 卷	答案错误	20	15	548	508	C++	2024-11-10 19:54:11
73122473	小歪商店故事: 卷	答案错误	20	15	556	500	C++	2024-11-10 19:51:51
73121491	小歪商店故事: 卷	答案错误	20	17	552	489	C++	2024-11-10 19:47:18
73120937	小歪商店故事: 卷	答案错误	20	16	536	483	C++	2024-11-10 19:44:54

- 神奇函数：在C和C++中，**toupper** 函数用于将小写字母转换为其对应的大写字母。这个函数定义在`<ctype.h>`或`<ctype.h>` 头文件中。
- 当某一个数据经常更新(比如说判断max值)我们可以将：
 - `if(i > max0) max0 = i` 改为 `max0 = max(max0,i)`
 - 同理：`if(i < min0) min0 = i` 改为 `min0 = min(min0,i)`
- 结构体在初始化 `vector<int>` 这类特殊特殊变量的时候，可以使用显式调用来初始化容器

```

1 struct st{
2     vector<int> a = vector<int>(6,0);
3 }

```

•

1 printf 和 scanf 语法初识

- **printf和scanf为格式化输出输入函数**
- **基本语法为**
- `printf("输出控制符",输出参数)`
- `scanf("输入控制符",输入参数)`

其中常用的输入(输出)控制符有：

%a(%A)	浮点数、十六进制数字和p-(P-)记数法(C99)
%c	字符
%d	有符号十进制整数
%f	浮点数(包括float和double)
%e(%E)	浮点数指数输出[e-(E-)记数法]
%g(%G)	浮点数不显示无意义的零"0"
%i	有符号十进制整数(与%d相同)
%u	无符号十进制整数

%o	八进制整数 e.g. 0123
%x(%X)	十六进制整数0f(0F) e.g. 0x1234
%p	指针
%s	字符串
%%	"%"

Note

- printf 和 scanf 都可以支持多位输入(输出)

scanf("%1d%1d",&a,&b) 就代表输入两个1位数字(两位数), 第一个赋值到a,第二个赋值到b (&为取址符)

- scanf 读取字符串的时候不用加寻址符
- scanf不能读取 *string str*格式的字符串
- **例子:** 我们可以用 printf & scanf 实现数字位数的获取来简化优化程序
- 优化前

```
1  #include<bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      float a = 114.5;
7      cin >> a ;
8      int b = a*10;
9      float c = b%10 + 0.1*((b/10)%10) + 0.01*((b/100)%10) + 0.001*(b/1000);
10     cout << c;
11
12     system("pause");
13     return(0);
14 }
```

- 优化后

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  int main()
4  {
5      int a;int b;int c;int d;
6      scanf("%1d%1d%1d.%1d",&a,&b,&c,&d);
7      printf("%1d.%1d%1d%1d",d,c,b,a);
8      system("pause");
9      return(0);
10 }
```

2 取整函数

- 头文件<cmath>

函数名称	函数说明
floor()	不大于自变量的最大整数
ceil()	不小于自变量的最小整数
round()	四舍五入到最邻近的整数
fix()	朝零方向取整

- floor() 朝负无穷方向取整
- ceil() 朝正无穷方向取整
- round() 函数，才是我们需要的四舍五入的函数，因为它会返回离自变量最近的整数，这个返回的整数可能大于也可能小于原来的数，但是一定是离它最近的那个整数。
- fix() 朝零方向取整，正数向下去，负数向上取

3 位运算(简单版)

- $i \ll 1$ 等同于 $i * 2$
- $i \gg 1$ 等同于 $i / 2$

4 sort()排序函数

- sort() 可以用一行实现数组的排序，而且可以实现从小到大，从大到小(甚至个位数从小到大之类的排序)的排序
- sort()函数的语法为 `sort(begin, end, cmp)` ,其中begin指向待sort数组的第一个元素的指针，end指向待sort数组的最后一个元素的下一个位置的指针
- cmp参数为排序准则，cmp参数可以不写，如果不写的话，默认从小到大进行排序
- 如果我们想从大到小排序可以将cmp参数写为 `greater<int>()` 就是对int数组进行排序，当然 `<>` 中我们也可以写 `double`、`long`、`float`等等

例：从小到大

```

1  int main()
2  {
3      int num[10] = {5,8,9,7,6,8,4,2,7,6};
4      sort(num,num+10);
5      for(int i=0;i<10;i++)
6      {
7          cout<<num[i]<<" ";
8      }
9  }
```

例：从大到小

```

1  int main()
2  {
3      int num[10] = {5,8,9,7,6,8,4,2,7,6};
4      sort(num,num+10,greater<int>());
5      for(int i=0;i<10;i++)
6      {
7          cout<<num[i]<<" ";
8      }
9  }

```

• cmp的编写规则

- 1. cmp函数的返回值是bool值
- 2. cmp传入的参数为(待排序的第一个类型& a, 待排序的第二个类型& b)
- 3. cmp里 `return a > b` 指降序排序 (从大到小)
- 例给无法排序的map排序

```

1  bool cmp(pair<int, int>& a, pair<int, int>& b){
2      return a.first > b.first;
3  }
4
5  int main()
6  {
7      map<int,int> arr;
8      vector<pair<int, int>> temp(arr.begin(),arr.end());
9  }

```

其实cmp的排序可以这么理解：

```

1  bool cmp(const <Type T> &a , const <Type T> &b ){
2      return a > b; //降序排列
3      //.....
4  }

```

💡 Tip

bool 类型的函数返回的是一个bool值，所以return回去的是一个bool值，传入的参数是 a,b 如果bool值为 1，则说明这个a b的顺序无需改变，如果传入的是0，则说明需要改变

注意：实际上cmp的规则并非如此，当我们不做任何判断直接返回1的话，数组会被倒序排序，而0则是没有任何改变

5 __gcd求最大公约数函数

格式： `__gcd(a,b)` 返回值为a, b的最大公因数

头文件： `#include< algorithm>`

实现：

```

1  int main()
2  {

```

```

3     int a,b,r;
4     cin >>a>>b;
5     //求x 和 y 的最大公约数，就是求 y 和 x % y 的最大公约数
6     while (a%b!=0) //判断a能否整除b
7     {
8         //开始循环找数
9         //判断余数能否被被除数整除
10        //循环到1
11        r=a%b;
12        a=b;
13        b=r;
14    }
15    cout << b;
16    return 0;
17 }

```

6 字符与字符串

- 字符串本质是一个数组，因此我们可以用str(字符串)[下标]的方式访问字符串的子字符，而下标从"0"开始计数
- 字符串的两种表示方式中，scanf()和printf()都无法访问 `string str`² 形式的字符串
- 可以用 `str.size()`; 的方式访问字符串长度(注意字符串最后会存在一个空字符，所以实际长度会比str.size输出的长度多一)
- 使用强转函数 `to_string` 可以时整型变为字符串型
- 我们可以使用 `str.empty()` 来判断一个字符串是否为空，若为空，该函数会返回一个 *True* 的bool值，否则返回 *False*
- `str.clear()` 可以帮我们愉快的删掉有效字符(但str.clear不会改变底层空间(capacity)的大小)
- 我们可以用 `str.capacity()` 的方式查看字符串的底层空间大小
 - 其实字符串(string类型)采用动态数组作为底层实现，它会为字符串提前预留一些额外的储存空间来减少内存的分配与释放次数

7 vector 容器 / 动态数组

- 使用 `vector<int> vec`³ 的方式来创建一个动态数组
- 对于 **vector**的赋值，不能直接使用`cin >> vec[i]` 的方式，我们可以建立一个临时变量，用 `vec.push_back(temp)` 的方法输入值

💡 Tip

9.17修改：对于使用 `vector<int> dp(a,b)` 形式的vec可以使用cin输入

- 对 **vector**容器的排序，我们必须使用迭代器来确定其数据位置，不能使用"+"确定位置

所以我们可以这样输入语法

```
sort(vec.begin(),vec.end())
```

- 使用 `vec[i]` 来获取第 `i - 1` 个元素(从零计数)

8 栈(stack)

- 栈是一种线性储存结构，其元素遵守先进后出的规则
- 只能在栈顶进行元素的添加和删除(进栈和出栈)
- 使用 `stack<int> st` 创建一个栈
- 对栈常见的操作有：
 - `empty()`: 判断栈是否为空栈，如果为空栈返回 `true`，否则或者 `false`
 - `push()`: 进栈操作，将新的元素放入到栈中，新的元素成为栈顶元素。
 - `pop()`: 出栈操作，栈顶元素从栈中离开
 - `top()`: 获取栈顶元素，但是不会移除它
 - `size()`: 获取栈的长度，即栈中元素的数量

9 宏常量定义小寄巧

打勾的就是好用的

☒ `#define endl "\n"`

☐ `#define int long long`

⚠ Warning

使用这个时候 `int main()` 要改为 `signed main()`

☐ `#define double long double`

10 数组前导零与后导零的删除

- 有的时候我们会用 `vis` 记录某一数据的增量和变化，到最后进行增量的排序
- 但是这个时候 `vis` 里没有在数据范围里的数据就是 0 这给我们 `sort` 数组带来极大的困惑

```
1 while(*arr.begin() == 0) arr.erase(arr.begin());
2 while(*(arr.end()-1) == 0) arr.erase(arr.end()-1);
```

- 我们可以使用这样的代码删除数组中的前后导零

11 位运算抽象版

📢 Important

位运算虽然在卡常时可以发挥一些优化作用，但其会导致代码可读性下降至一个难以理解的程度

- 用位运算代替 `*=2` `/=2` 的操作

- `int a = 10; (a <= 2) == (a*=2); (a >= 2) == (a/=2)`

💡 Tip

对位运算来说，左移右移都是改变二进制位的操作，比如我们可以这么理解

设某一数 x

$$x_2 = tttttt$$

$$x \ll s = tttttt \underbrace{0000 \dots 00}_s$$

$$\text{则 } x_{10} = x \times 2^s$$

同理，右移就是 $x_{10} = \frac{x}{2^s}$

- 用位运算代替swap()

```
1  int a = 5 , b = 2;
2  a ^= b, b ^= a, a ^= b;
3  //swap(a,b)
```

- ### 💡 Tip

$\wedge$ 运算，即**异或(XOR)**运算，比较两个值的二进制位，如果两个值相同，则结果为假，如果两个值不同，则结果为真

推理：

1. **第一次异或运算：** `A = A ^ B`

- 这一步将 `A` 和 `B` 的值进行异或运算，并将结果存储在 `A` 中。
- 此时，`A` 包含了 `A` 和 `B` 的异或结果，而 `B` 仍然是原来的值。

2. **第二次异或运算：** `B = A ^ B`

- 由于 `A` 现在包含了 `A` 和 `B` 的异或结果，我们将这个结果与 `B` 进行异或运算，并将结果存储在 `B` 中。
- 这一步实际上是将 `B` 的原始值与 `A` 和 `B` 的异或结果进行异或运算，这将导致 `B` 现在包含 `A` 的原始值。

3. **第三次异或运算：** `A = A ^ B`

- 现在 `B` 包含了 `A` 的原始值，我们将 `A`（包含 `A` 和 `B` 的异或结果）与 `B`（现在包含 `A` 的原始值）进行异或运算。
- 这一步将导致 `A` 现在包含 `B` 的原始值。

- 取出二进制的某一位

```
1  int a = 15;
2  for(int i = 31; i >= 0 ; --i) cout << (x >> i & 1);
```

怎么实现的呢？

从31位开始向前将a的每一个二进制位与 1 做与位操作

与位操作即：

- 如果两个比较的位都是1，则结果位是1。
- 如果两个比较的位中至少有一个是0，则结果位是0。

那么与 1 做&操作的二进制位则必然会等于其本身

- 优化版:

- ```
1 vector<int> arr;
2 int a = 15;
3 for(int i = 31; i >= 0 ; --i) arr.push_back(a >> i & 1);
4 while(!arr.begin()) arr.erase(arr.begin());
5 for(auto &&i : arr) cout << i;
```

- 实现了前导零的删除

- 用异或判断两变量是否相等

- ```
1 int a = 10, b = 5;
2 if(a^b) cout << "不相等";
3 else cout << "相等";
```

- 使用^48实现int和char的转化

- ```
1 int a = 6;
2 char ch = x^48;
3 cout << ch << endl;
4 //同理char变int也可以用^48
5 int y = ch^48;
6 cout << y;
```

- 用 &1 判断函数奇偶性

- ```
1 int a = 7;
2 if(a&1)
3 {
4     cout << "奇数" << endl;
5 }
6 else
7 {
8     cout << "偶数" << endl;
9 }
```

12 时间函数帮你计算运算时间

```
1 clock_t st = clock();
2 //代码.....
3 clock_t ed = clock();
4 cout << "time: " << ed - st << " ms" << endl;
```

13 一组数的各种数计算

- 对一组数据而言, 对其分布有影响的数据类型有: → 平均数, 中位数

平均数:

```
1 int sum = 0;
2 for(int i = 0; i < arr.size() ; i++){
3     sum += arr[i];
4 }
5 double tnum = (sum/n*1.0);
```

中位数:

```
1 vector<int> arr(n);
2 double cent = 0;
3 sort(arr.begin(),arr.end());
4 if(arr.size()%2 == 1) cent = arr[n/2 + 1];
5 else cent = (arr[n/2 + 1] + arr[n/2]*1.0)/2;
6 //数组从0开始技术的时候，要注意下标减一
7 if(arr.size()%2 == 1) cent = arr[n/2 + 1 - 1];
8 else cent = (arr[n/2 + 1 - 1] + arr[n/2 - 1]*1.0)/2;
```

算法

算法大纲(登神长阶)

1 顺序表

- ☐ 线性枚举
- ☒ 前缀和
- ☐ 双指针
- ☐ 二分枚举
- ☐ 三分枚举
- ☐ 离散化
- ☒ 冒泡排序
- ☐ 选择排序
- ☒ 快速排序
- ☐ 插入排序
- ☐ 希尔排序
- ☐ 归并排序
- ☐ 堆排序
- ☐ 基数排序
- ☐ 计数排序
- ☐ 模拟
- ☐ 贪心

2 链表

- ☒ 单向链表
- ☐ 双向链表

3 栈

- ☒ LIFO栈（后进先出）
- ☐ 单调栈

4 队列

- ☐ FLFO队列（先进先出）
- ☐ 双端队列
- ☐ 单调队列

5 字符串

- ☐ KMP
- ☐ 字典树
- ☐ 马拉车
- ☐ AC自动机
- ☐ 后缀数组
- ☐ BM

6 树

- ☐ 二叉树
- ☐ 二叉搜索树
- ☐ AVL树
- ☐ 线段树
- ☐ 霍夫曼树
- ☐ 堆
- ☐ 红黑树
- ☐ 伸展树
- ☐ 左偏树
- ☐ Treap B+树
- ☐ 树链剖分

7 图

- ☐ 二分图
- ☐ 最短路
- ☐ 最小生成树
- ☐ 最近公共祖先

- ☐ 深度优先搜索
- ☐ 强连通分量
- ☐ 双连通分量
- ☐ 2-sat
- ☐ 欧拉回路
- ☐ 哈密尔顿回路
- ☐ 迭代加深
- ☐ 广度优先搜索
- ☐ 拓扑排序
- ☐ A*
- ☐ 稳定婚姻
- ☐ 双向广搜
- ☐ 查分约束
- ☐ 并查集
- ☐ 哈希表
- ☐ 跳跃表
- ☐ 树状数组
- ☐ 最大流

8 动态规划

- ☒ 递推
- ☐ 线性DP
- ☐ 记忆化搜索
- ☐ 背包问题
- ☐ 树形DP
- ☐ 区间DP
- ☐ 数位DP
- ☐ 状压DP

1 递归

递归的定义：**函数的自我调用**

例：

- **利用递归实现阶乘**

```
1  #include<bits/stdc++.h>
2  using namespace std;
3
4  int out(int n)
5  {
6      int res;
7      if (n == 1)
```

```

8      {
9          res = 1;
10     }
11     else
12     {
13         res = out(n-1)*n; //在这里又调用了一次out,即out(n-1) = out(n-2)*(n-1)
14     }
15
16     return res;
17 }
18
19 int main()
20 {
21     cout << out(5);
22     return 0;
23 }

```

- 基于递归的算法

- [DFS](#)

学递归有感而发：

- 只有上帝和出题人知道递归传参究竟是怎么传的

- 递归的缺陷

使用递归计算斐波那契数列第n项($n < 50$)

```

1  int f(int n)
2  {
3      int res;
4      if (n == 1 || n == 2)
5      {
6          res = 1;
7      }
8      else
9      {
10         res = f(n-1)+f(n-2);
11     }
12
13     return res;
14 }
15
16 int main()
17 {
18     for (size_t i = 1; i <= 50; i++)
19     {
20         cout << i << "    " << f(i) << endl;
21     }
22
23     return 0;

```

- 运行不难发现，在第46项以后，运行极为缓慢
- 由此引入 **递推算法**

2 递推

- 从1开始，向下求解，直到输出正确函数
- 用若干重复计算解决实际问题的方法
- 找规律构成递推式

```

1  #include<bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      long arr[60] = {0};
7      arr[1] = 1;
8      arr[2] = 1;
9      for (size_t i = 3; i <= 50; i++)
10     {
11         arr[i] = arr[i-1] + arr[i-2];
12     }
13     for (size_t i = 1; i <= 50; i++)
14     {
15         cout << i << "    " << arr[i] << endl;
16     }
17
18     return 0;
19 }
```

- 又找到一道递推好题 [洛谷P1044\[栈\]](#)
- 上代码：

```

1  /*
2      P1044递推解法
3      注意到待入栈数n里的第k个数：设其方案有f[k] (k > 0)
4      则再设 k 前面的数的排列方式有 f[k-1] 种
5      k 后面的数的排列方式有 f[n-k]种
6      根据组合数原理：f[k] = f[k-1] * f[n-k]
7      则有 f[n] = sum_{k = 0}^{n-1} f[k] 即 f[n] = f[0]f[n-1] + f[1]f[n-2]
      ..... + f[n-1]f[0]
8      简单分析不难发现f[0] = 1 , f[1] = 1, f[2] = 2;
9  */
10
11 #include <bits/stdc++.h>
12 #define endl "\n"
13 using namespace std;
14
15 int main()
16 {
```



```

17     int n;
18     long long f[20] = {1,1,2};
19     cin >> n;
20     //在这里 i 表示上文的 n ; j 表示上文的 k;
21     //因为f[0] , f[1] , f[2] 均已明确 i 从 3 开始算;
22     for (size_t i = 3; i <= n; i++)
23     {
24         for (size_t j = 1; j <= i; j++)
25         {
26             f[i] += f[j-1]*f[i-j];
27         }
28     }
29     cout << f[n];
30     return 0;
31 }

```

❗ Important

主要在于对 N 里的任何一项 k 存在: $f(k) = f(k-1) \times f(n-k)$

$f(k)$ 表示第 k 项的情况:

则 $f(N) = \sum_{k=1}^{N-1} f(k)$

即: $f(n) = f(0)f(n-1) + f(1)f(n-2) + \dots + f(n-1)f(0)$

3 贪心

- 每一步再选择中都选择当前状态下的最优解
- 通过局部最优解做到全局最优解
- 例:

贪心

题: 过河的最短时间

描述: N 位旅行者在夜里过桥需要借助手电筒。但 N 个人中只有一个手电筒,而且桥同时只能让两个人过。每个人单独过桥所需时间已知,但如果两个人同时过桥则所需时间是走得慢的那个人单独过桥所需的时间。设计一个方案,让这 N 个人尽快过桥,计算这 N 个人的最短过桥时间。

比如:有甲乙丙丁四个人,他们过河所需的时间分别是1, 2, 5, 10。让最快的2个人先过桥,然后让跑的最快的人回去接剩下的人。先让甲乙过去(2分钟),甲回来(1分钟),甲丙过去(5分钟)甲回来(1分钟),甲丁再过去(10分钟),总共需要19分钟。

输入: 第一行 N (表示有 N 个人要过河)
第二行是 N 个整数 S ,表示这 N 个人过河所需时间 ($0 < S < 100$)

输出: 所有人过河的最短时间

样例输入: 4

1 2 5 10

样例输出: 17

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int N ,sum = 0,p = 1,j =0,emp = 0;
7      cin >> N;
8      int arr[N] = {0};
9      int arrp[N] = {0};

```

```

10     for (size_t i = 1; i <= N; i++)
11     {
12         cin >> arr[i];
13     }
14     sort(arr+1, arr+N+1);
15
16     if (N == 1)
17     {
18         cout << arr[1];
19         return 0;
20     }
21
22     while (1)
23     {
24         if (N - j == 2 || N - j == 1)
25         {
26             sum += arr[2];
27             break;
28         }
29         sum += arr[1] + 2*arr[2] + arr[N-j];
30         j += 2;
31     }
32     cout << sum;
33
34     return 0;
35
36 }

```

① Note

- 这题的题解找个时间我再写

分数背包问题 (Fractional Knapsack Problem)

- 代表题目：洛谷P2240[部分背包问题][<https://www.luogu.com.cn/problem/P2240>]
部分背包问题，本质甚至不是动态规划，而是贪心，这个题名出的非常具有迷惑性
 - 特点：
 1. 有一个容量为 T 的背包
 2. 有 N 组物品，每组物品分别有以下两个特性：
 1. 价值：这堆物品所代表的价值
 2. 重量：这堆物品的重量
 3. 与经典的0/1背包不同的是，分数背包允许将物品划分为重量为1的单位物品
- 策略：先计算每一堆物品的单位价值，再根据单位价值装填背包。

对P2240代码如下：

```

1  // #pragma GCC optimize(3)
2  #include <bits/stdc++.h>
3  // #define int LL
4  #define endl '\n'
5  #define size_t int
6  #define all(v) v.begin(), v.end()

```

```

7  using namespace std;
8  typedef long long LL;
9  typedef vector<int> vint;
10 typedef vector<vint> vvint;
11 typedef vector<string> vstr;
12 typedef pair<int, int> pii;
13 typedef vector<pii> vpii;
14
15 bool cmp(pair<double, int> a, pair<double, int> b)
16 {
17     if (a.first == b.first) return a.second > b.second;
18     return a.first > b.first;
19 }
20 signed main()
21 {
22     //ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
23     int n, t;
24     cin >> n >> t;
25     vector<pair<double, int>> coin(n);
26     for (auto &&[val, wei] : coin) {
27         double m, v;
28         cin >> m >> v;
29         val = (v / m);
30         wei = m;
31     }
32     sort(all(coin), cmp);
33     double ans = 0;
34     for (size_t i = 0; i < n; i++) {
35         if (t >= coin[i].second) {
36             ans += coin[i].second * coin[i].first;
37             t -= coin[i].second;
38         }
39         else {
40             ans += coin[i].first * t;
41             break;
42         }
43     }
44     printf("%.2f", ans);
45     return 0;
46 }

```

不难发现这其实是一种贪心而非背包问题

4 桶排序

- 有一种排序方式可以很快的对数字进行排序

```

1  #include<bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {

```

```

6     int arr[11];
7     for (size_t i = 1; i <= 10; i++)
8     {
9         int a;
10        cin >> a;
11        arr[a]++;
12    }
13    for (size_t j = 0; j <= 10; j++)
14    {
15        if (arr[j] != 0)
16        {
17            for (size_t i = 1; i <= arr[j]; i++)
18            {
19                cout << j << " ";
20            }
21        }
22    }
23
24    return 0;
25 }

```

📌 Important

- 在使用桶排序解决实际题目的时候，一定要注意初始开始值
- 点名表扬洛谷 [P5729 【深基5.例7】工艺品制作]

📌 Note

- 桶排序的思想可以用在需要标记类的题目上，例如洛谷 [P1047 [NOIP2005 普及组] 校门外的树] [P5729 【深基5.例7】工艺品制作]

5 高精度

- 一般而言，在long long 格式下的字符占用有8个字节，其范围是 $-2^{63} \sim 2^{63}-1$ (19位数)这个区间，那么，超过这个区间的计算我们又该如何进行呢？

5.1 高精度加法

- 在进行超过19位数相加的大数加法的时候，我们可以模拟竖式加法的原理，对数组进行操作

如图

高精度算法

$$\begin{array}{r} \text{" } 7 \text{ } 8 \text{ } 9 \text{ " } \\ + \quad \text{" } 5 \text{ } 6 \text{ " } \\ \hline 7 \quad 13 \quad 15 \\ 7 \quad 14 \quad 5 \\ 8 \quad 4 \quad 5 \end{array}$$

$a1$

9	8	7	
---	---	---	--

$a2$

6	5		
---	---	--	--

$a3$

15	13	7	
----	----	---	--

1. 使用字符串获取大数字
2. 将字符串的数字提取出来逆序储存在数组中
3. 对数组中的数组做加法并存储到另一个数组中
4. 逆序输出数组

```
1  #include<bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      string s1,s2;
7      int a1[210],a2[210],a3[210] = {0};
8      getline(cin,s1);
9      getline(cin,s2);
10     for (size_t i = 0; i < s1.size(); i++)
11     {
12         a1[s1.size()-i-1] = s1[i] - '0';
13     }
14     for (size_t i = 0; i < s2.size(); i++)
15     {
16         a2[s2.size()-i-1] = s2[i] - '0';
17     }
18     int len = max(s1.size(),s2.size());
19     for (size_t i = 0; i < len;i++)
20     {
21         a3[i] = a1[i] + a2[i];
22     }
23     for (size_t i = 0; i < len; i++)
24     {
25         if (a3[i] >= 10)
26         {
27             a3[i + 1] = a3[i]/10;
28             a3[i] = a3[i]%10;
```

```

29     }
30 }
31 if (a3[len] != 0)
32 {
33     len++;
34 }
35 for (int i = len - 1; i >= 0; i--)
36 {
37     cout << a3[i];
38 }
39 return 0;
40 }

```

- 更好用的字符串 **string**类型的高精度

```

1 string largeadd(string& a, string& b)
2 {
3     if (a.size() <= b.size()) swap(a, b);
4     int p = 0;
5     for (size_t i = 0; i < b.size(); i++) {
6         int ai = a[a.size() - i - 1] - '0';
7         int bi = b[b.size() - i - 1] - '0';
8         int sum = ai + bi + p;
9         if (sum >= 10) {
10             p = 1;
11             sum -= 10;
12         } else p = 0;
13         a[a.size() - 1 - i] = sum + '0';
14     }
15     for (size_t i = b.size(); i < a.size(); i++) {
16         int ai = a[a.size() - i - 1] - '0';
17         if (ai == '9' && p == 1) {
18             a[a.size() - i - 1] = '0';
19         } else {
20             a[a.size() - i - 1] = ai + p + '0';
21             p = 0;
22         }
23     }
24     if (p == 1) a.insert(a.begin(), '1');
25     return a;
26 }

```

5.2 高精度减法

- 与加法类似，主要是注意借位与进位的不同
- 负数的处理

```

1 string largemin(string a, string b)
2 {
3     int flag = 0;
4     if (b.size() >= a.size() && b >= a) {
5         swap(a, b);
6         flag = 1;
7     }

```

```

8     int p = 0;
9     for (size_t i = 0; i < b.size(); i++) {
10         int ai = a[a.size() - 1 - i] - '0';
11         int bi = b[b.size() - 1 - i] - '0';
12         int diff = ai - bi - p;
13         if (diff < 0) {
14             p = 1;
15             diff += 10;
16         } else p = 0;
17         a[a.size() - i - 1] = diff + '0';
18     }
19     for (size_t i = b.size(); i < a.size(); i++) {
20         int ai = a[a.size() - i - 1] - '0';
21         if (ai == 0 && p == 1) {
22             a[a.size() - i - 1] = '9';
23         } else {
24             a[a.size() - i - 1] = ai - p + '0';
25             p = 0;
26         }
27     }
28     while (*a.begin() == '0' && a.size() > 1)
29         a.erase(a.begin());
30     if (flag) a.insert(a.begin(), '-');
31     return a;
32 }

```

5.3 高精度乘法

💡 Tip

对于乘法来说，高精度的最佳计算思维就是将两个数拆分，用一个数的个十百位依次去乘以另一个数

拿我们常用的，主要的步骤有以下几点：

1. 字符串化数字，倒置字符串
2. 两位相乘，计算结果与储存位数
3. 数组变字符串

```

1  // #pragma GCC optimize(2)
2  #include <bits/stdc++.h>
3  #define endl '\n'
4  using namespace std;
5
6  string largemulti(string a, string b)
7  {
8      if (a == "0" || b == "0") return "0";
9      int len1 = a.size();
10     int len2 = b.size();
11     vector<int> result(len1 + len2, 0);
12     for (int i = len1 - 1; i >= 0; i--) {
13         for (int j = len2 - 1; j >= 0; j--) {
14             int mult = (a[i] - '0') * (b[j] - '0');
15             int sum = result[i + j + 1] + mult;
16             result[i + j + 1] = sum % 10;

```

```

17         result[i + j] += sum / 10;
18     }
19 }
20 string res = "";
21 for (size_t i = 0; i < result.size(); i++) {
22     if (!(result[i] == 0 && res.empty())) {
23         res += to_string(result[i]);
24     }
25 }
26 return res;
27 }
28
29 /*
30     1 5  --> a
31     * 2 1 --> b
32     -----
33     1 5  --> i == 0
34     3 0 | --> i == 1
35     | | |
36     v v v
37     -----
38     3 1 5 --> result --> res
39 */
40
41 int main()
42 {
43     ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
44     cout << largemulti("123", "234");
45     return 0;
46 }

```

6 STL

- 优点: 更加简短的代码语句, 调试方便
- 缺点: 有些时候用更复杂的方式进行算法实现

6.1 容器

6.1.1 [vector](#)

构造

一维数组: `vector<类别> dp(长度, 初值)` ⁴

二维数组: `vector<vector<int>> dp(行数, vector<int> (列数, 初值));`

三维数组:

```

1 | vector<vector<vector<int>>> dp3(3层, vector<vector<int>>(行数, vector<int> (列
   | 数, 0)));

```


等价于 `int mat[][][]`

Note

我们也可以使用 `vector<vector<int>> dp(100,vector<int>())` 来构建不指定列数的二维数组

6.1.1.1 尾接与尾删

- 尾接: `dp.push_back(x)` [解释: 在dp数组末尾添加数字x]
- 尾删: `dp.pop_back(x)` [解释: 在dp数组末尾删除数字x]

6.1.1.2 size函数

- `dp.size()` [解释: 获取dp数组的长度(数组内有多少个数)]

6.1.1.3清空数组

- `dp.clear()` [清空数组内数据]

6.1.1.4 empty函数

使用 `dp.empty()` 判断数组是否为空, 空返回true(1), 非空返回false(0)

一般这个函数会放在if语句中

```
1  if(dp.empty()) //如果数组为空, 则执行语句
2  {
3      //....
4  }
```

[!!!] 6.1.1.5 resize函数

- `dp.resize(m,n)` [m表示新大小, n表示: 如果增加长度, 多出来的位置的默认数字]
- 注意resize函数改小的话, 会将多出来的数据删除

6.1.1.6 访问vector的数据

- 使用`dp[x]`, 访问dp数组内x-1的数据

6.1.1.7 vector的赋值与读取

1. 一维数组的赋值

- 方法一(推荐)

```
1  vector<int> dp(10,0);
2  for(int i = 0; i < k; i++)
3  {
4      int temp;
5      cin >> temp;
6      dp.push_back(temp);
7  }
```

- 方法二(不推荐)

```

1 vector<int> dp(10,0);
2 for(int i = 0; i < k; i++)//k不能大于10(k > 10 也可以读入dp,但是会有诡异的bug)
3 {
4     cin >> dp[i];
5 }

```

Note

所以动态读写一套下来为:

```

1 vector<int> dp(0,0);
2 for (size_t i = 0; i < 15; i++)
3 {
4     int temp;
5     cin >> temp;
6     dp.push_back(temp);
7 }
8
9 for (size_t i = 0; i < dp.size(); i++)
10 {
11     cout << dp[i] <<" ";
12 }

```

2. 二维数组动态读写

Note

```

1 vector <vector<int>> dp;
2 vector<int> dp1;
3 for (int i = 0; i <k; i++)
4 {
5     for (int j = 0; j <p; j++) //内部数组保存
6     {
7         int value;
8         cin >> value;
9         v.push_back(dp1);
10    }
11    dp.push_back(dp1); //保存dp1的每个元素到dp[i]中
12    dp1.clear(); //清空dp1内元素
13 }
14
15 for (int i = 0; i <array.size(); i++)
16 {
17     for (int j = 0; j < p; j++)
18     {
19         cout <<array[i][j];
20     }
21     cout<<endl;
22 }
23 return 0;

```

- 思路: 先建立动态二维数组dp和动态临时一维数组dp1
- dp1负责保存单行数据

- dp负责保存dp1保存下来的行数据从而形成多数据
- 记得clear dp1的元素

6.1.1.8 vector的使用情况

- 例: $n \times m$ 的矩阵, $1 \leq n, m \leq 10^6$ 且 $n \times m \leq 10^6$
- 普通数组就是 `int arr[100010][100010]`, 直接炸内存(MLE)
- 动态数组就可以 `vector<vector<int>> dp(n+10, vector<int>(m+10, 0))`
- 在读取了m,n后再设立数组, 解决了炸内存的尴尬
- 虽然有时候我也会用 `int arr[m+10][n+10]` 来写数组(好孩子不要学)
- vector储存在堆空间, 不会炸栈

6.1.1.9 注意事项

- 提前规定长度
- vector的push_back逻辑是, 当超过长度时会消耗时间进行重分配

```

1 // 优化前: 522ms
2 vector<int> a;
3 for (int i = 0; i < 1e8; i++)
4 {
5     a.push_back(i);
6 }
7 // 优化后: 259ms
8 vector<int> a(1e8);
9 for (int i = 0; i < a.size(); i++)
10 {
11     a[i] = i;
12 }

```

2. 小心size_t溢出

vector 获取长度的方法 `.size()` 返回值类型为 `size_t`, 通常 OJ 平台使用的是 32 位编译器 (有些平台例如 cf 可选 64 位), 那么该类型范围为 $[0, 2^{32})$.

```

1 vector<int> a(65536);
2 long long a = a.size() * a.size(); // 直接溢出变成0了

```

6.1.1.10 和其他容器的组合技

- 和pair二元组

```

1 vector<pair<int,int>> dp1(10);
2 pair<int,int> p1;
3 for (size_t i = 0; i < 5; i++)
4 {
5     cin >> p1.first >> p1.second;
6     dp1[i] = p1;
7 }
8 int k;
9 cin >> k;
10 for (size_t i = 0; i < dp1.size(); i++)

```

```

11 {
12     if (k == dp1[i].first)
13     {
14         cout << dp1[i].second;
15     }
16 }

```

Tip

`dp[i]`可以作为一个二元组绑定死.first和.second

dp容器可以起到结构体的作用

6.1.2 栈 [stack](#)

头文件: <stack>

6.1.2.1 构造方式

`stack<double> stk` [stk是栈名];

6.1.2.2 进栈与出栈及取栈顶部

- 进栈: `stk.push(x)`; [将x放入栈中]
- 出栈: `stk.pop()`; [栈顶出栈]
- 取栈顶: `stk.top()` [获取栈顶]

6.1.1.3 用vector模拟stack

使用 `dp.back()` 取栈(容器)顶

6.1.1.4 写栈的注意事项

- 不能访问栈的内部元素
- 下面都是错误用法

```

1 stack<int> stk;
2 for(int i = 1; i < stk.size(); i++)
3 {
4     cout << stk[i] << endl;
5 }
6 for(auto ele : stk)
7 {
8     cout << stk << endl;
9 }

```

6.1.2.5 与 vector 相比 stack的优势是什么?

- stack效率是高于vector的
- stack的内存占用更低
- 在某些算法实现下(如深度优先搜索), stack可能是更自然的选择

6.1.3 队列 [queue](#)

- `incload <queue>`

通过二次封装双端队列，实现先进先出(双端获取)的数据结构

6.1.3.1常用方法

作用	用法	示例
构造	<code>queue<类型> que</code>	<code>queue<int> que</code>
进队	<code>que.push(元素)</code>	<code>que.push(1)</code>
出队	<code>.pop()</code>	<code>que.pop()</code>
取队首	<code>.front()</code>	<code>int a = que.front()</code>
取队尾	<code>.banc()</code>	<code>int a = que.back()</code>
查看大小	<code>.size()</code>	<code>int a = que.size()</code>
清空	<code>.clear()</code>	<code>que.clear()</code>
判空	<code>.empty()</code>	<code>que.empty()</code>

6.1.3.2 注意事项

不能访问内部元素!示例同6.1.2.3

6.1.4 优先队列(堆) [priority_queue](#)

`include <queue>`

6.1.4.1 构造

`priority_queue<类型,容器,比较器>`

- 类型:要储存的数据类型
- 容器:储存数据的底层容器,默认为 `vector<T>`,竞赛时保存默认即可
- 比较器: 比较大小使用的比较器,默认为 `less<T>`,可以自定义

```
1 priority_queue<int> pqe1;  
2 priority_queue<int,vector<int>,greater<int>> pqe2;//变小顶堆
```

自定义比较器尽量不用,容易犯迷糊

6.1.4.2 常用语法

作用	用法	示例
进堆	<code>.push(元素)</code>	<code>que.push(1);</code>
出堆	<code>.pop()</code>	<code>que.pop();</code>
取堆顶(最大值/最小值)	<code>.top()</code>	<code>int a = que.top();</code>

作用	用法	示例
查看大小/判空	和vector一致	略

Note

进出堆复杂度 $O(\log n)$,取堆顶 $O(1)$

适用场景

- 保持数据的有序性,每次向队列中插入大小不定的元素,或每次从队列取出最大/最小的元素,元素数量为 n ,插入操作数量为 k
 - 使用快排: $k \cdot n \log n$
 - 使用优先队列: $k \cdot \log n$

6.1.4.3 注意事项

- 仅堆顶可读

```
1 cout << qque[1] <<endl; //错误
```

- 所有元素不可写

```
1 qque[1] = 2;
2 pque.top() = 1;
3 //均为错误
```

但如果要修改堆顶元素

```
1 int tp = pque.top(); //保存堆顶
2 pque.pop(); //弹出堆顶
3 pque.push(tp + 1); //通过保存的堆顶修改堆顶
```

6.1.5 集合set

提供对数时间的插入、删除、查找的集合数据结构。底层原理是红黑树。

集合三要素	解释	set	multiset	unordered_set
确定性	一个元素要么在集合中,要么不在	✓	✓	✓
互异性	一个元素仅可以在集合中出现一次	✓	✗ (任意次)	✓
无序性	集合中的元素是没有顺序的	✗ (从小到大)	✗ (从小到大)	✓

6.1.5.1 常用操作

- 函数构造 `set<类型, 比较器>`
- 插入元素 `st.insert(元素)`
- 删除元素 `st.erase(元素)`
- 查找元素 `st.find(元素)`
- 查找元素个数 `st.count`

6.1.5.2 遍历

可以使用遍历器来遍历set数据:

```
1 set<int> st;  
2 for (set<int>::iterator it = st.begin() ; it != st.end(); ++it )  
3 {  
4     cout << *it << endl;  
5 }
```

基于范围的循环:

```
1 for(auto &ele : st)  
2 {  
3     cout << ele << endl;  
4 }
```

6.1.5.3 适用范围

- 元素的去重 `[1,1,2,3,3,5,7] → [1,2,3,5,7]`
- 元素顺序的维护 `[1,6,8,4,1] → [1,4,6,8]`
- 元素大小很大但数量很少的情况(大小: $[-10^{18}, 10^{18}]$, 数量 10^6)

6.1.5.4 注意事项

- set数据不存在下标的说法,但可以用遍历器找数据:

```
1 set<int>::iterator it = st.begin();  
2 advance(it, 2); //迭代器后面会讲  
3 cout << *it << endl;
```

- 元素都是只读的, set迭代器提取的元素都是只读的(因为是const迭代器), 不能够修改它的值, 需要先erase再insert

```
1 cout << *st.begin() << endl; //正确  
2 *st.begin() = 1; //错误, 不可写
```

- 不可用迭代器计算下标

set 的迭代器不能像 vector 一样相减得到下标。下面是错误用法:

```
1 auto it = st.find(2); // 正确, 返回2所在位置的迭代器。  
2 int idx = it - st.begin(); // 错误! 不可相减得到下标。
```

6.1.6 映射 `map`

```
include <map>
```

- 提供对数时间的有序键值对结构[任意类型的映射];

```
1 map<string,int> a;
2 a["qaz"] = 1;
3 a["abc"] = 2;
4 a["mmp"] = 3;
```

- `key[键]`的概念

```
1 a[key] = value;
```

`key` 在映射中处于中括号内,表示提示`map`的元素

- 性质

性质	解释	map	multimap	unordered_map
互异性	一个键仅可以在映射中出现一次	✓	✗ (任意次)	✓
无序性	键是没有顺序的	✗ (从小到大)	✗ (从小到大)	✓

6.1.6.1 构造及常用操作

```
map<key,value> mp;
```

- 增/改: `map[0] = 1`
- 倘若没有定义就直接访问`map`,就会返回一个初值(默认为0)
- 查找元素[找的是 `key`]: `mp.find()` → 返回这个元素的迭代器,若找不到,返回`mp.end()`(尾迭代器)
- 删除: `mp.erase(元素)`
- 查找[找的是 `key`]: `mp.count()` → 返回的是元素数量
- 清空判空同上

6.1.6.2 遍历

1. 萌新式遍历[适合<int int>型]

```
1 map<int , int> mp;
2 mp[6] = 3;
3 mp[5] = 1;
4 mp[7] = 3;
5 mp[9] = 666;
6 for (size_t i = 0; i < mp.size(); i++)
7 {
8     cout << mp[i] <<endl;
9 }
```


缺点很明显,给个图就明白了

```
0
0
0
0
0
1
3
3
0
666
请按任意键继续. . . |
```

2. 迭代器式遍历

```
1 string a;
2 map<string , int> mp;
3 mp["aaa"] = 1;
4 mp["bbb"] = 5;
5 mp["tsts"] = 3;
6 for (map<string , int>::iterator it = mp.begin() ; it != mp.end() ; it++)
7 {
8     cout << it->second << endl;
9 }
```

这个遍历器指向一个键对,所以得用 `it->first` 或 `it->second` 来判断指向的哪一个

3. auto范围遍历

```
1 string a;
2 map<string , int> mp;
3 mp["aaa"] = 1;
4 mp["bbb"] = 5;
5 mp["tsts"] = 3;
6 for(auto &el:mp)
7 {
8     cout << el.first << " " << el.second << endl;
9 }
```

6.1.6.3 适用范围

- 维护特殊的映射[string $\rightarrow$ int]

统计输入的字符串组中每个字符串出现的次数

```
1 map<string , int> mp;
2 vector<string> vec;
3 vec.push_back("aqa");
4 vec.push_back("aqa");
5 vec.push_back("qaq");
6 vec.push_back("qaq");
```

```

7  vec.push_back("wqw");
8  vec.push_back("aqa");
9  vec.push_back("qaq");
10
11  for (size_t i = 0; i < vec.size(); i++)
12  {
13      mp[vec[i]]++;
14  }
15  for (auto &el : mp)
16  {
17      cout << el.first << " " << el.second << endl;
18  }

```

```

aqa 3
qaq 3
wqw 1
请按任意键继续. . . |

```

效果如此

6.1.6.4 注意事项

- 空map会返回一个默认值
- 不能使用遍历器找下标

`unordered_map` 无序map 也称哈希表，我们可以随意的建立映射关系，时间复杂度是 $O(1)$

6.1.7 字符串string

6.1.7.1 常用方法

- 构造: `string str;`
- 输入: `cin >> str;`
- 输出: `cout << str;`
- string 的初值构造 `string str(100, '0')`
- 赋值 `str = "awa";`
- 判断相等 `str1 == str2`
- 修改字符 `str[0] = "a"`
- 连接字符串 `str1 + str2;`
- 字符串尾接 `str1 += "awa";`
- 取子串:

```

1  string s1 = "123123123";
2  cout << s1.substr(3) << endl; //从第三位开始输出到末尾
3  cout << s1.substr(3,4) << endl; //从第三位输出,输出4位

```

- 查找函数: `find(字符串)` → 返回字符串起始点的下标(若找不到,则会返回一个 `string::npos`)
- 对上面一条: 返回的是下标, 是`size_t`类型的数字!

6.1.7.2 string 转化

- $str \rightarrow int$: `int x = stoi(str)`
- $str \rightarrow long\ long$: `long long x = stoll(str)`
- $str \rightarrow float$: `stof()`
- $str \rightarrow double$: `stod()`
- $str \rightarrow long\ double$: `stold()`
- $int \rightarrow str$: `string str = to_string(x)`

6.1.7.3 注意事项

- 尾接要用 += [使用 `str = str + "awa"` 很慢]
- `.substr()` 方法下,第一个参数传的是字符串起点下标,第二个是字符串长度
- `.find()` 的实现是暴力枚举,复杂度是 $O(n^2)$

6.1.8 二元组 [pair](#)

构造

```
pair<int, int> pr;
```

6.1.8.1 常用方法

- 赋初值: `pair<int, int> pr = {1,2}`
- 老式: `pair<int, int> pr2 = make_pair(1,2)`
- 判同: `pr == pr2`
- 三元组曲线救国法: `pair<pair<int, char>, char> p3;`
- 访问第一个值 `.first`
- 访问第二个值 `.second`

6.1.8.2 适用范围

适用于所有需要二元组的场景,效率和自己定义结构体差不多

6.1.9 列表 [list](#)

构造

```
list<类型> lt
```

6.1.9.1 优势与适用范围

list容器插入和删除元素的效率较高,时间复杂度为常数级别,其底层为带头双向循环链表

6.1.9.2 常用方法

- 定义:
 - 构造空 `list()` / 含有n个元素的类型容器 `list` / 拷贝某个类型容器的复制品

```
1 list<int> lt1; //构造int类型的空容器
2 list<int> lt2(10,2); //构造含有10个2的int类型容器
3 list<int> lt3(lt2); //拷贝构造int类型的lt2容器的复制品
4 list<int> lt4{ 1,2,3,4,5 }; // 直接使用花括号进行构造---C++11允许
```

- 迭代器复制字符内容

```
1 string s("hello world");
2 list<char> lt5(s.begin(),s.end()); //构造string对象某段迭代器区间的内容
```

6.1.9.3 list的遍历及迭代器的操作

- 迭代器
- 正向迭代器

```
1 int arr[] = {1,1,4,5,1,4}; //构造数组
2 list<int> lt(arr,arr+sizeof(arr)/sizeof(arr[0])); //copy数组到list
3 for(list<int>::iterator it = lt.begin();it != lt.end();++it)
4 {
5     cout << *it <<endl;
6 }
```

- 反向遍历器(抽象)[防止你不知道]

```
1 int arr[] = {1,1,4,5,1,4};
2 list<int> lt(arr,arr+sizeof(arr)/sizeof(arr[0]));
3 for(list<int>::reverse_iterator it = lt.rbegin();it != lt.rend();++it)
4 {
5     cout << *it <<endl;
6 }
```

- 范围for(好用)

```
1 int arr[] = {1,1,4,5,1,4};
2 list<int> lt(arr,arr+sizeof(arr)/sizeof(arr[0]));
3 for (auto &i : lt)
4 {
5     cout << i <<endl;
6 }
```

- 常见容器操作

- `.size()`:返回容器中有效元素的个数
- `.resize()`:调整容器的有效元素大小(size)
- `.empty()`:判断容器是否为空
- `.clear()`:用于清空容器,清空后容器的size为0,但是头结点(哨兵位)不会被清除
- `.resize(a,b)` 有两个参数:
 - a:将list大小变为(a)
 - b:若list新大小大于原大小,则新增的大小用b填充

6.1.9.4 list容器的常见访问操作

- `.front()` :访问list头元素[返回list的第一个元素]
- `.back()` :访问list尾元素[返回list的最后一个元素]

6.1.9.5 list 容器的常见修改操作

函数(接口)名称	函数(接口)说明
<code>.push_front()</code>	在list首元素前插入元素
<code>.pop_front()</code>	删除list首元素
<code>.push_back()</code>	在list尾部插入元素
<code>.pop_back()</code>	删除list最后一个元素
<code>.insert()</code>	在 <code>list<int>::iterator it</code> 前插入元素
<code>.erase()</code>	在 <code>list<int>::iterator it</code> 前删除元素
<code>.swap()</code>	交换两个元素

Important

- 有关`insert()`的操作: ⁵
- `insert`共有三种形式:
 - `insert(iterator, value);`
 - `insert(iterator, num, value);`
 - `insert(iterator, iterator1, iterator2);`
- `insert`的所有操作都由迭代器位置确定,不存在`lt[2]`这类的中括号表操作
- 对 `insert(iterator, value);` (会返回一个新迭代器指向插入的元素)

```
1 //创建一个数组
2 int arr[] = {1,1,4,5,1,4};
3 list<int> lt(arr,arr+sizeof(arr)/sizeof(arr[0]));
4 cout << "befor" << endl;
5 for (auto &i : lt)
6 {
7     cout << i <<endl;
8 }
9 //创建一个迭代器指向lt的头元素
10 list<int>::iterator it = lt.begin();
11 //迭代器向后移动2位,指向4
12 advance(it,2);
13 //在4的迭代器前,插入元素3
14 auto itnew = lt.insert(it,3);
15 cout << "after" << endl;
16 for (auto &i : lt)
17 {
18     cout << i <<endl;
19 }
20 cout << "new iterator = " << *itnew <<endl;
```

```
21 return 0;
```

```
befor
1
1
4
5
1
4
after
1
1
3
4
5
1
4
new iterator = 3
```

- 对 `insert(iterator, num, value);` (会返回一个新迭代器指向插入的第一个元素)

```
1 //创建一个数组
2 int arr[] = {1,1,4,5,1,4};
3 list<int> lt(arr,arr+sizeof(arr)/sizeof(arr[0]));
4 cout << "befor" << endl;
5 for (auto &i : lt)
6 {
7     cout << i <<endl;
8 }
9 //创建一个迭代器指向lt的头元素
10 list<int>::iterator it = lt.begin();
11 //迭代器向后移动2位,指向4
12 advance(it,2);
13 //在4的迭代器前,插入元素3个3
14 auto itnew = lt.insert(it,3,3);
15 cout << "after" << endl;
16 for (auto &i : lt)
17 {
18     cout << i <<endl;
19 }
20 //新迭代器的位置在第一个3
21 cout << "new iterator = " << *itnew <<endl;
22 return 0;
```

```

befor
1
1
4
5
1
4
after
1
1
3
3
3
4
5
1
4
new iterator = 3
请按任意键继续. . . |

```

- 对 `insert(iterator, iterator1, iterator2);`

```

1 //创建一个数组
2 int arr[] = {1,1,4,5,1,4};
3 list<int> lt(arr,arr+sizeof(arr)/sizeof(arr[0]));
4 cout << "befor" << endl;
5 for (auto &i : lt)
6 {
7     cout << i <<endl;
8 }
9 //创建一个迭代器指向lt的头元素
10 list<int>::iterator it = lt.begin();
11 //迭代器向后移动2位,指向4
12 advance(it,2);
13 //建立新的list或(vector);
14 vector<int> lt2 = {1,9,1,9,8,1,0};
15 //确定迭代器位置
16 auto it1 = lt2.begin();
17 auto it2 = lt2.end();
18 //在4的迭代器前,插入迭代器it1 - it2 这之间的数
19 //会返回插入的数的第一个元素的迭代器
20 auto itnew = lt.insert(it,it1,it2);
21 cout << "after" << endl;
22 for (auto &i : lt)
23 {
24     cout << i <<endl;
25 }
26 cout << "new iterator = " << *itnew;
27 return 0;

```

```

befor
1
1
4
5
1
4
after
1
1
1 ← 返回的迭代器位置
9
1
9
8
1
0 插入的数据
4
5
1
4
new iterator = 1

```

- `.erase()` 用法和 `.insert()` 差不多,只是从添加元素变为删除元素删除的为迭代器指向的元素或两个迭代器之间(包括本身)的元素

6.1.9.6 list容器常用的操作

函数声明	接口说明
<i>splice</i>	将元素从列表转移到其它列表
<i>remove</i>	删除具有特定值的元素
<i>remove_if</i>	删除满足条件的元素
<i>unique</i>	删除重复值
<i>sort</i> (慢)	容器中的元素排序
<i>merge</i>	合并排序列表
<i>reverse</i>	反转元素的顺序

- `.splice()` 一共有四种形式
 - `splice(iterator_pos, otherList)`: 将otherList中的所有元素移动到iterator_pos指向元素之前

```

1 list<int> ls1 = {1,2,3,4,5};
2 list<int> ls2 = {10,20,30,40,50};
3 ls2.splice(ls1.begin(),ls2); //和ls1.splice(ls1.begin(),ls2); 等价
4
5 for(auto &p : ls1)
6 {
7     cout << p << " ";
8 }
9 //输出 10 20 30 40 50 1 2 3 4 5
10 //此时ls2的情况: 空

```

- `splice(iterator_pos, otherList, iter1)`: 从 otherList转移 iter1 指向的元素到当前list。元素被插入到 iterator_pos指向的元素之前。


```

1  list<int> ls3 = {10,20,30};
2  list<int> ls4 = {3,5,7,8};
3  auto it = ls3.begin();
4  advance(it,1);
5  ls3.splice(it,ls4,ls4.begin());
6  for(auto &p : ls3)
7  {
8      cout << p << " ";
9  }
10 cout << endl;
11 //ls4
12 for(auto &p : ls4)
13 {
14     cout << p << " ";
15 }
16 //输出:
17 //10 3 20 30
18 //5 7 8

```

- ***splice(iterator_pos, otherList, iter_start, iter_end)***: 从 otherList 转移范围 [iter_start, iter_end) 中的元素到 当前列表。元素被插入到 iterator_pos 指向的元素之前。

```

1  list<int> ls5 = {1,2,3,4,5};
2  list<int> ls6 = {10,20,30,40,50};
3  auto it2 = ls6.begin();
4  advance(it2,2);
5  auto it3 = ls5.begin();
6  auto it4 = ls5.end();
7  advance(it3,1);
8  advance(it4,-2);
9  ls6.splice(it2,ls5,it3,it4);
10 for(auto &p : ls6)
11 {
12     cout << p << " ";
13 }
14 cout << endl;
15 //ls5
16 for(auto &p : ls5)
17 {
18     cout << p << " ";
19 }
20 cout << endl;
21 //输出:
22 // 10 20 2 3 30 40 50
23 // 1 4 5

```

deque [双向队列](#)

- **作用**: deque 可以作为一个双向队列，在队首队尾以及任意位置实现元素的插入和删除
- **定义**: 和 vector 一致

成员函数:

```

1  push_back() //在队列的尾部插入元素。

```

```

2  emplace_front()//与push_front()的作用一样
3  push_front()//在队列的头部插入元素。
4  emplace_back()//与push_back()的作用一样
5  pop_back()//删除队列尾部的元素。
6  pop_front()//删除队列头部的元素。
7  back()//返回队列尾部元素的引用。
8  front()//返回队列头部元素的引用。
9  clear()//清空队列中的所有元素。
10 empty()//判断队列是否为空。
11 size()//返回队列中元素的个数。
12 begin()//返回头位置的迭代器
13 end()//返回尾+1位置的迭代器
14 rbegin()//返回逆头位置的迭代器
15 rend()//返回逆尾-1位置的迭代器
16 insert()//在指定位置插入元素
17 erase()//在指定位置删除元素

```

- deque可以通过forrange / iterator / 下标 遍历

6.2 迭代器(遍历器)

- 概念：迭代器是一种检查容器内元素并遍历元素的数据类型，通常用于对C++中各种容器内元素的访问，但不同的容器有不同的迭代器，初学者可以将迭代器理解为指针。
- 迭代器可以干嘛？

```

1  int main()
2  {
3      vector<int> arr;
4      for (size_t i = 0; i < 10; i++)
5      {
6          arr.push_back(i);
7      }
8      for(vector<int>::iterator it = arr.begin(); it != arr.end();it++)
9      {
10         cout << *it <<endl;
11     }
12
13     return 0;
14 }

```

我们观察上面这个程序，这是它的输出结果

0
1
2
3
4
5
6
7
8
9

请按任意键继续 . . . |

不难发现，这个数组被遍历输出了

- `for(vector<int>::iterator it = arr.begin(); it != arr.end(); it++)` 这一行
- `vector<int>::iterator it = arr.begin();` → 创立了一个迭代器，`vector<int>::iterator` 表明创立了一个vector迭代器 `it` 是迭代器名称 `arr.begin()` 是数组的头迭代器
- 迭代器之间也可以用比较运算符 `==` or `!=`
- 迭代器也可以使用自增运算 → `it++`

常用容器的迭代器

- **vector —— 随机访问迭代器**
- **deque —— 随机访问迭代器**
- **list —— 双向迭代器**
- **set / multiset —— 双向迭代器**
- **map / multimap —— 双向迭代器**
- **stack —— 不支持迭代器**
- **queue —— 不支持迭代器**

下面主要讲解随机访问迭代器 双向迭代器

- 双向迭代器

```
1 void text()
2 {
3     list<int> lst;
4     for (int i = 0; i < 10; ++i)
5     {
6         lst.push_back(i);
7     }
8     list<int>::iterator it; //创建list的迭代器
9     cout << "遍历lst并打印: ";
10    for (it = lst.begin(); it != lst.end(); ++it) //用 != 比较两个迭代器
11    {
12        cout << *it << " ";
13    }
14    //此时it=lst.end(), 这个位置是最后一个元素的下一个位置，没有存储数据
15    --it; //等价于it--, 回到上一个位置
16    //it -= 1; //报错, 虽然都是-1, 但这种方式是随机迭代器才有的功能
```

```

17     cout << "\n1st的最后一个元素为: " << *it << endl;
18 }

```

• 随机迭代器

```

1  void text()
2  {
3      vector<int> v;
4      for (int i = 0; i < 10; ++i)
5      {
6          v.push_back(i);
7      }
8      vector<int>::iterator it;
9      for (it = v.begin(); it != v.end(); ++it) //用 != 比较两个迭代器
10     {
11         cout << *it << " ";
12     }
13     cout << endl;
14     for (it = v.begin(); it < v.end(); ++it) //用 < 比较两个迭代器
15     {
16         cout << *it << " ";
17     }
18     cout << endl;
19     it = v.begin(); //让迭代器重新指向首个元素的位置
20     while (it < v.end()) //间隔一个输出
21     {
22         cout << *it << " ";
23         it += 2; // 用 += 移动迭代器
24     }
25     cout << endl;
26
27     it = v.begin();
28     cout << it[5] << endl; //用[]访问
29 }

```

Important

- 对vector数组迭代器来说,

迭代器的辅助函数

STL 中有用于操作迭代器的三个函数模板, 它们是:

- `advance(it, n);` 使迭代器 `it` 向前或向后移动 `n` 个元素。
- `distance(it1, it2);` 计算两个迭代器之间的距离, 即迭代器 `it1` 经过多少次 `++` 操作后和迭代器 `it2` 相等。如果调用时 `it1` 已经指向 `it2` 的后面, 则这个函数会陷入死循环。
- `iter_swap(it1, it2);` 用于交换两个迭代器 `it1`、`it2` 指向的值。
要使用上述模板, 需要包含头文件

```

1  #include<algorithm>

```

6.3 STL函数

对容器操作类

- `sort(iterator_begin, iterator_end, cmp)` 快速排序
- `find`: 顺序查找。 `find(iterator_begin, iterator_end, value)`, 其中 `value` 为需要查找的值。
- `reverse`: 翻转数组、字符串。 `reverse(iterator_begin, iterator_end())` 或 `reverse(a + begin, a + end)`。
- `unique`: 去除容器中相邻的重复元素。 `unique(ForwardIterator first, ForwardIterator last)`, 返回值为指向 **去重后** 容器结尾的迭代器, 原容器大小不变。与 `sort` 结合使用可以实现完整容器去重。
- `move`: 可以高效赋值容器, 当你确定某一个容器在后面不需要被使用时可以使用 `move` 来降低时间复杂度, 尤其是对 `vector<pair<int, int>>` 这类复杂容器而言

```
1  for(auto &&i : f){
2      set<int> temp = dp;
3      for(auto &&j : dp){
4          temp.emplace(i+j);
5      }
6      dp = move(temp);
7  }
```

这是一个求一个数组取任意个数个数字相加的板子, 其中使用到 `move(temp)` 就起到了降低时间复杂度的作用

对容器改动类

- `() .emplace()` `() .emplace_back()`
 - `emplace` 是 C++11 引入的标准容器函数, 用于直接在容器中**构造对象**, 而不是先创建对象再插入。它适用于几乎所有 STL 容器 (如 `vector`, `set`, `map`, `deque` 等), 提供了比 `insert` 更高效的方式。
 - 例子: 我们可以直接在 `set<pair<int, int>>` 后插入 (x,y)

```
1  set<pair<int, int>> st;
2  //传统办法
3  st.insert(make_pair(x,y));
4  //emplace办法
5  st.emplace(x,y);
```

- 对 `vector` 而言, `emplace_back()` 几乎可以完全代替 `push_back()` 而 `emplace` 则能代替 `insert`

```
1  vector<pair<int, int>> v
2  v.push_back(make_pair(x,y)) == v.emplace_back(x,y);
3  v.insert(v.begin(), make_paie(x,y)) == v.emplace(v.begin(), x,y)
```

7 快速幂

- 不多说，先上算法

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long fastpower(long long a , long long b)
5  {
6      long long ans = 1;
7      while (b > 0)
8      {
9          if (b&1)
10         {
11             ans *= a;
12         }
13         a *= a;
14         b >>= 1;
15     }
16     return ans;
17 }
18
19 int main()
20 {
21     cout << fastpower(2,8);
22 }
```

快速幂的核心思路在下面这串代码里

```
1  while (b > 0)
2  {
3      if (b&1)
4      {
5          ans *= a;
6      }
7      a *= a;
8      b >>= 1;
9  }
```

Important

- b 是我们的指数，只要大于零，我们便把这个循环继续下去
- 对任何一个数，都可以拆解为2进制的数，快速幂的核心思想在于，在幂次 b 二进制转化为十进制的时候 0×2^k 是可以不用乘进结果的而 1×2^k 是需要被乘进结果的
- 示例：
- 假设我们要计算 a^{13} ：
 - 13 的二进制表示是 1101_2 。
 - 从最低位开始：
 - 第一位 (1)：需要 a^{2^0} 。
 - 第二位 (0)：不需要 a^{2^1} 。
 - 第三位 (1)：需要 a^{2^2} 。

- 第四位 (1) : 需要 a^{2^3} 。

因此, 我们可以计算:

$$\begin{aligned}a^{13} &= a^{1 \times 2^0} + a^{0 \times 2^1} + a^{1 \times 2^2} + a^{1 \times 2^3} \\&= a^{1+0+4+8} \\&= a^{13}\end{aligned}$$

- 只要b大于零, 我们就把他的二进制右移(除二), 这样我们就能获取下一位二进制数字
- 如果b的二进制位为1, 我们就把该位置的幂次方乘进ans中, 待b移位完成后ans也完成了幂运算
- 快速幂的时间复杂度为 $O(\log N)$

8 dfs 深度优先搜索

- 题目来源: [P1036 \[NOIP2002 普及组\] 选数](#)
- 题目核心:

[NOIP2002 普及组] 选数

题目描述

已知 n 个整数 $x_1, x_2, \dots, x_n$, 以及 1 个整数 k ($k < n$)。从 n 个整数中任选 k 个整数相加, 可分别得到一系列的和。例如当 $n = 4$, $k = 3$, 4 个整数分别为 3, 7, 12, 19 时, 可得全部的组合与它们的和为:

$$3 + 7 + 12 = 22$$

$$3 + 7 + 19 = 29$$

$$7 + 12 + 19 = 38$$

$$3 + 12 + 19 = 34$$

💡 Tip

- 有别于传统模拟, 这种在 N 个数里找 k 个数的操作, 正常人应该都不会想到使用 循环 或 枚举 但受限于知识则停滞不前
- 其实思路很简单, 选多少个数就建立多少个标记, 然后从某一标记开始移动, 将所有标记过的数加和即可
- 问题是, 怎么使用代码实现?

DFS:深度优先搜索

这时候就需要用到递归搜索了。

该类搜索算法的特点在于, 将要搜索的目标分成若干「层」, 每层基于前几层的状态进行决策, 直到达到目标状态。

先看核心代码

```

1 void dfs(int m, int sum, int startx){
2     if(m == k){
3         if(isprime(sum))
4             ans++;
5         return ;
6     }
7     for(int i = startx; i < n; i++){
8         dfs(m + 1, sum + a[i], i + 1);
9     }
10 }

```

前置知识：每次递归调用 `dfs` 时，都会创建一个新的栈帧，并将 `m + 1`、`sum + a[i]` 和 `i + 1` 作为参数传递给 `dfs` 函数。当 `dfs` 函数执行到 `return` 语句时，它会返回到上一个栈帧，也就是上一次调用 `dfs` 的地方

• 例子

- 让我们用一个简化的例子来说明DFS算法的运行原理。假设我们有一个数组 `a = [1, 3, 5, 7]` 和 `k = 2`，我们要找出所有长度为2的子数组，其和为素数

```

1  开始
2  |
3  v
4  dfs(0, 0, 0)  <- 初始化, m=0 (子数组长度), sum=0 (子数组和), startx=0 (起始索引)
5  |
6  |
7  |--> dfs(1, 1, 1)  <- 选择a[0], m=1, sum=1, startx=1
8  |  |
9  |  |--> dfs(2, 4, 2)  <- 选择a[1], m=2, sum=4, startx=2
10 |  |  |
11 |  |  |--> 检查sum=4 (不是素数), 结束这个分支
12 |  |
13 |  |<--- 返回到 dfs(1, 1, 1) [返回到这个栈帧的时候for循环内的参数不会变化] //保留了
    |  |  这个栈帧的数据
14 |
15 |  |--> dfs(2, 8, 2)  <- 选择a[2], m=2, sum=8, startx=2
16 |  |  |
17 |  |  |--> 检查sum=8 (不是素数), 结束这个分支
18 |  |
19 |  |<--- 返回到 dfs(1, 1, 1)
20 |  |
21 |<----- 返回到 dfs(0, 0, 0)
22 |
23 |--> dfs(1, 3, 1)  <- 选择a[1], m=1, sum=3, startx=1
24 |  |
25 |  |--> dfs(2, 6, 2)  <- 选择a[2], m=2, sum=6, startx=2
26 |  |  |
27 |  |  |--> 检查sum=6 (不是素数), 结束这个分支
28 |  |
29 |  |<--- 返回到 dfs(1, 3, 1)
30 |
31 |  |<--- 返回到 dfs(0, 0, 0)
32 |
33 |--> dfs(1, 5, 1)  <- 选择a[2], m=1, sum=5, startx=1

```



```

34 | | |
35 | | |--> dfs(2, 10, 2)  <- 选择a[3], m=2, sum=10, startx=2
36 | | |
37 | | | |--> 检查sum=10（不是素数），结束这个分支
38 | | |
39 | | |<-- 返回到 dfs(1, 5, 1)
40 | |
41 | | |<-- 返回到 dfs(0, 0, 0)
42 | |
43 | |<-- 返回到开始
44 | 结束

```

使用DFS实现全排列

- DFS的核心思路是一路往下寻找，不撞南墙不回头说明当DFS到达边界情况时，就完成了一次搜索
- 使用DFS时就要考虑完成一次“搜索”所需要的条件和边界情况

使用DFS实现全排列

- 思考第一步：全排列的一次情况的边界条件
 - 我们不妨对DFS传入一个参数 `step` 表示完成一次全排列的步骤
 - 当 `step` 到 `n` 的时候，一次全排列的一种情况便结束了：
 - 在当前情况下，我们需要做的事情是：
 - 输出全排列的一次情况
- Q1：如何输出一次全排列情况？
 - A:用result数组存
- 思考第二步：全排列的实现
 - 我们可以使用一个 `path` 来保留加入到result数组
 - `path` 数组在一次查找过程中先将其标记，再进入下一个DFS函数查找，再取消标记，即一次回溯操作

```

1  vector<bool> path;
2  vint result;
3  int t = 1;
4
5  void quan(int step)
6  {
7      if (step == t + 1) {
8          for (size_t i = 1; i <= t; i++) {
9              cout << result[i] << " ";
10             }
11             cout << endl;
12         }
13         for (size_t i = 1; i <= t; i++) {
14             if (path[i] == 0) {
15                 path[i] = 1;
16                 result[step] = i;
17                 quan(step + 1);
18                 path[i] = 0;
19             }
20         }

```

```

21     return;
22 }
23
24 signed main()
25 {
26     //ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
27     cin >> t;
28     path = vector<bool>(t + 10);
29     result = vint(t + 10);
30     quan(1);
31     return 0;
32 }

```

[板子] 一组数据取任意个数数据进行操作

例1: 在数组[1,2,3,4,5] 中取任意个数, 求这些取出来的数相加的结果

方法一: DFS爆搜

```

1  // #pragma GCC optimize(3)
2  #include <bits/stdc++.h>
3  // #define int LL
4  #define endl '\n'
5  #define size_t int
6  #define all(v) v.begin(), v.end()
7  using namespace std;
8  typedef long long LL;
9  typedef vector<int> vint;
10 typedef vector<vint> vvint;
11 typedef vector<string> vstr;
12 typedef pair<int, int> pii;
13 typedef vector<pii> vpii;
14
15 vint res;
16 void dfs(int n, int sum, int T, vint k)
17 {
18     if (n >= T) return;
19     sum += k[n];
20     res.emplace_back(sum);
21     for (size_t i = n + 1; i < T; i++) {
22         dfs(i, sum, T, k);
23     }
24 }
25
26 signed main()
27 {
28     //ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
29     int T = 1;
30     cin >> T;
31     vint k(T);
32     for (auto &&i : k) {
33         cin >> i;
34     }
35     for (size_t i = 0; i < T; i++) {
36         dfs(i, 0, T, k);

```

```

37     }
38     cout << 0 << " ";
39     for (auto &&i : res) {
40         cout << i << " ";
41     }
42     return 0;
43 }

```

其结果表现为：

- 0 1 3 6 10 15 11 7 12 8 4 8 13 9 5 10 6 2 5 9 14 10 6 11 7 3 7 12 8 4 9 5
- 充分体现了人类看不懂栈帧的特点

法二：DP

```

1  // #pragma GCC optimize(3)
2  #include <bits/stdc++.h>
3  // #define int LL
4  #define endl '\n'
5  #define size_t int
6  #define all(v) v.begin(), v.end()
7  using namespace std;
8  typedef long long LL;
9  typedef vector<int> vint;
10 typedef vector<vint> vvint;
11 typedef vector<string> vstr;
12 typedef pair<int, int> pii;
13 typedef vector<pii> vpii;
14
15 signed main()
16 {
17     // ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
18     int T;
19     cin >> T;
20     vint food(T);
21     for (auto &&i : food) {
22         cin >> i;
23     }
24     vint dp;
25     dp.emplace_back(0); // 初始状态
26     for (auto &&i : food) {
27         vint temp = dp;
28         for (auto &&j : dp) {
29             temp.emplace_back(i + j);
30         }
31         dp = move(temp);
32     }
33     for (auto &&i : dp)
34     {
35         cout << i << " ";
36     }
37     return 0;
38 }

```

输出如下：

0 1 2 3 3 4 5 6 4 5 6 7 7 8 9 10 5 6 7 8 8 9 10 11 9 10 11 12 12 13 14 15

- 已验证，两个程序的结果除了顺序完全一致

例2: [P2036](#) [COCI2008-2009 #2] PERKET

例二 法一: DFS

```
1  // #pragma GCC optimize(3)
2  #include <bits/stdc++.h>
3  #define int LL
4  #define endl '\n'
5  #define size_t int
6  #define all(v) v.begin(), v.end()
7  using namespace std;
8  typedef long long LL;
9  typedef vector<int> vint;
10 typedef vector<vint> vvint;
11 typedef vector<string> vstr;
12 typedef pair<int, int> pii;
13 typedef vector<pii> vpii;
14
15 int T;
16 vpii food;
17 // vint result;
18 int ans = 1e6;
19
20 void dfs(int n, int sum_s, int sum_k)
21 {
22     if (n == T) {
23         sum_s *= food[n - 1].first;
24         sum_k += food[n - 1].second;
25         ans = min(ans, abs(sum_s - sum_k));
26         return;
27     }
28     sum_s *= food[n].first;
29     sum_k += food[n].second;
30     ans = min(ans, abs(sum_s - sum_k));
31     for (size_t k = 0; k < T; k++) {
32         for (size_t i = n; i < T; i++) {
33             dfs(i + 1, sum_s, sum_k);
34         }
35         sum_s = 1;
36         sum_k = 0;
37     }
38 }
39
40 signed main()
41 {
42     //ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
43     cin >> T;
44     food = vpii(T);
45     // result = vint(T, 0);
46     for (auto &&[s, k] : food) {
```

```

47     cin >> s >> k;
48 }
49 dfs(0, 1, 0);
50 cout << ans;
51 return 0;
52 }
53 // 比我命还暴力这个算法
54 // 这题绝对能用DP写，待我研究一下

```

例二 法二：DP

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef long long ll;
5  typedef pair<ll, ll> pii;
6
7  signed main()
8  {
9      int T = 1;
10     cin >> T;
11     vector<pii> food(T);
12     for (auto &&[s, k] : food) {
13         cin >> s >> k;
14     }
15     // 使用集合记录所有可能的（酸度，苦度）组合
16     set<pii> dp;
17     dp.emplace(1, 0); // 初始状态
18     for(auto &[s, b] : food){
19         set<pii> temp = dp;
20         for(auto &[acid, bitter] : dp){
21             temp.emplace(acid * s, bitter + b);
22         }
23         dp = move(temp);
24     }
25     // gtp写的，确实很精巧，用set记录了每个可能的情况
26     // 因为初始状况是(1,0),就相当于每一次内层循环的第一次都是只选当前组的食物
27     // 每一次都会把dp数组过完一遍，相当于在之前的所有情况下加一个 选择当前食物的情况
28     ll result = LLONG_MAX;
29     for(auto &[acid, bitter] : dp){
30         if(acid != 1 || bitter != 0){
31             result = min(result, abs(acid - bitter));
32         }
33     }
34     cout << result;
35     return 0;
36 }

```

9 素数筛

9.1 一般双重筛

- 通过不断试除来判断某一数字 k 有无因数
- 时间复杂度为 $O(N^2)$

```
1  bool prime(long long i)
2  {
3      if (i == 2) return 1;
4      else if (i == 1) return 0;
5      for (size_t k = 2; k * k <= i ; k++)
6      {
7          if (i%k == 0)
8          {
9              return 0;
10         }
11     }
12     return 1;
13 }
14
15 int main()
16 {
17     long long n;
18     cin >> n;
19     for (size_t i = 2; i <= n; i++)
20     {
21         if (prime(i))
22         {
23             cout << i <<endl;
24         }
25     }
26
27     return 0;
28 }
```

9.2 埃拉托斯特尼筛法

- 核心思路是先标记素数，然后把素数的所有倍数全标记为非素数
- 时间复杂度是 $O(n \log \log n)$
- 对于任意一个大于1的正整数 n ,那么它的 x 倍就是合数($x > 1$)。利用这个结论，我们可以避免很多次不必要的检测。

如果我们从小到大考虑每个数，然后同时把当前这个数的所有（比自己大的）倍数记为合数，那么运行结束的时候没有被标记的数就是素数了。

```
1  vector<int> prime;
2  vector<bool> IsPrime(1e7);
3
4  void Eratosthenes(long long n)
5  {
6      IsPrime[0] = IsPrime[1] = 0; //前两项不为素数，记作0
7      for (size_t i = 2; i <= n; i++) //先全部记作1
```

```

8      {
9          IsPrime[i] = 1;
10     }
11     //memset(IsPrime,1,sizeof(IsPrime)); //似乎使用memset的时间复杂度也是O(N)
12     for (size_t i = 2; i <= n; i++)
13     {
14         //素数筛选
15         //从第三项开始, 如果这个数被标记为1, 就把它记作素数, 放入数组
16         //同时从 i*i 项开始, 每次将 i 的倍数标记为非素数
17         if (IsPrime[i])
18         {
19             prime.push_back(i);
20             if ((long long)i * i > n) continue; //超过n的不计
21             for (size_t j = i*i; j <= n; j += i)
22             {
23                 IsPrime[j] = 0;
24             }
25         }
26     }
27 }
28
29 int main()
30 {
31     long long n;
32     cin >> n;
33     Eratosthenes(n);
34     for (auto &&i : prime) cout << i <<endl;
35     return 0;
36 }

```

- 我们可以只筛选到 $\sqrt{N}$ 来降低操作次数

```

1  #include <bits/stdc++.h>
2  #define endl "\n"
3  using namespace std;
4
5  vector<int> prime;
6  vector<bool> IsPrime(1e7);
7
8  void Eratosthenes(long long n)
9  {
10     IsPrime[0] = IsPrime[1] = 0; //前两项不为素数, 记作0
11     for (size_t i = 2; i <= n; i++) //先全部记作1
12     {
13         IsPrime[i] = 1;
14     }
15     //memset(IsPrime,1,sizeof(IsPrime)); //似乎使用memset的时间复杂度也是O(N)
16     for (size_t i = 2; i*i <= n; i++)
17     {
18         //素数筛选
19         //从 i*i 项开始, 每次将 i 的倍数标记为非素数
20         if (IsPrime[i])
21         {
22             for (size_t j = i*i; j <= n; j += i)
23             {

```

```

24         IsPrime[j] = 0;
25     }
26 }
27 }
28 //计入数组
29 for (size_t i = 2; i <= n; i++)
30 {
31     if (IsPrime[i])
32     {
33         prime.push_back(i);
34     }
35 }
36 }
37
38 int main()
39 {
40     long long n;
41     cin >> n;
42     Eratosthenes(n);
43     for (auto &&i : prime) cout << i <<endl;
44     return 0;
45 }

```

- 我们也可以改写0/1来实现非初始化bool数组

```

1  #include <bits/stdc++.h>
2  #define endl "\n"
3  using namespace std;
4
5  vector<int> prime;
6  vector<bool> IsPrime(1e7);
7
8  void Eratosthenes(long long n)
9  {
10     IsPrime[0] = IsPrime[1] = 0; //前两项不为素数，记作0
11     for (size_t i = 2; i*i <= n; i++)
12     {
13         //素数筛选
14         //从 i*i 项开始，每次将 i 的倍数标记为非素数
15         if (!IsPrime[i])
16         {
17             for (size_t j = i*i; j <= n; j += i)
18             {
19                 IsPrime[j] = 1;
20             }
21         }
22     }
23     //计入数组
24     for (size_t i = 2; i <= n; i++)
25     {
26         if (!IsPrime[i])
27         {
28             prime.push_back(i);
29         }
30     }

```



```

31 }
32
33 int main()
34 {
35     long long n;
36     cin >> n;
37     Eratosthenes(n);
38     for (auto &&i : prime) cout << i << endl;
39     return 0;
40 }

```

9.3 欧拉筛法

- 时间复杂度 $O(n)$
- 埃氏筛法仍有优化空间，它会将一个合数重复多次标记。我们可以用改进的筛法欧拉筛来计算

```

1  #include <bits/stdc++.h>
2  #define endl "\n"
3  using namespace std;
4
5  vector<int> prime;
6  vector<bool> not_prime(1e7);
7
8  void Euler(long long &n)
9  {
10     for (size_t i = 2; i <= n; i++)
11     {
12         //如果该数标记为 0 即 非(非素数),计入素数数组
13         if (!not_prime[i])
14         {
15             prime.push_back(i);
16         }
17         // 换言之, i 之前被 prime[j] 筛过了
18         // 由于 prime 里面质数是从小到大的, 所以 i 乘上其他的质数的结果一定会被
19         // prime[j] 的倍数筛掉, 就不需要在这里先筛一次, 所以这里直接 break掉
20         for (auto &&j : prime)
21         {
22             if(i*j > n) break;
23             not_prime[i*j] = 1;
24             if(i%j == 0) break;
25         }
26     }
27 }
28
29 int main()
30 {
31     long long n;
32     cin >> n;
33     Euler(n);
34     for (auto &&i : prime) cout << i << endl;
35     return 0;
36 }

```

10 DP动态规划[uncomplete]

背包DP

- 两个特点：
 1. 一个物体有 大小 + 价值
 2. 有一个背包只能容纳某一个特点大小
- 要求：找最大价值
- **一般性转移方程**——> 设背包大小为 N , 有 Y 个物品, 物品价值为 val_i 物品大小为 $size_j$
- 设 $dp[i][j]$ 表示任取 $0 \rightarrow i$ 个物品在 j 容量下取得的 $\sum a_k$ 的最大值
- **初始化**见下:

```
1 vector<vector<int>> dp(Y, vector<int>(N+1, 0));
2 //对[0]行的初始
3 for (size_t i = size[0]; i <= N; i++) {
4     dp[0][i] = val[0];
5 }
6 //对[0]列的初始
7 //在背包初始大小为0的情况, 只有 size = 0 的物品可以被装下
8 for(size_t j = 0; j <= Y ; j++){
9     dp[i][0] = 0;
10 }
```

- **现在我们来推递推方程:**
- 对于一个 $dp[i][j]$ 而言表示的是在 j 背包大小下取得 前 i 个 val 的最大值
- 从 $dp[i-1][j]$ 到 $dp[i][j]$ 有两种情况
 1. $j < size_i \implies dp[i][j] = dp[i-1][j]$ 即当前容量装不下 $size_i$ 这个物品, 只能与前一个的价值相同 $\implies$ 即增加的 $val = 0$
 2. $j > size_i$:
 1. 装 $size_i$: $dp[i][j] = dp[i][j - size[i]] + val[i]$.1
 2. 不装 $size_i$: $dp[i][j] = dp[i-1][j]$
- **对 .1 的解释:** 选择装下大小为 $size_i$ 价值为 val_i 的物品, 那对 dp 从 $[i-1][j]$ 到的 $[i][j]$ 而言:
 - i 表示选取的范围 那显然 $i-1 \rightarrow i$
 - j 表示背包容量大小: 既然我们选取了这个物品 i , 那我们就看对于没有该物品时候的背包大小所表示的价值 + 这个物品 i 的价值即: $j \rightarrow j - size_i$
 - $dp[i][j]$ 本质是最大价值, 则我们需要加上所选物品的价值 即: $dp[i][j - size[i]] + val[i]$
- 在选择 $j > size_i$ 的两种情况下选择较大的一方表示最大价值
- 即完整递推表达式为:

$$dp[i][j] = \begin{cases} dp[i-1][j] & j < size[i] \\ \max(dp[i-1][j], dp[i][j - size[i]] + val[i]) & j \geq size[i] \end{cases}$$

代码表示见下

```

1  for (size_t i = 1; i < m; i++) {
2      for (size_t j = 1; j <= t; j++) {
3          if (j < size[i])
4              dp[i][j] = dp[i - 1][j];
5          else
6              dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - size[i]] + val[i]);
7      }
8  }

```

-

11 状态压缩

状态压缩指的是用较小的数据结构（如整数或位掩码）来表示一个可能的状态或组合，从而减少存储空间和计算复杂性

- 当两个数据分析特别大的时候，我们可以考虑使用二进制来实现0/1的分类;
- 下面举例使一个数组的分为 a b两组,找到 $\max(\sum a, \sum b)$ 中的最小值
- a 代表一个分类 b 代表一个分类

```

1  cin >> n;
2  for(int i = 0; i < n; i++) cin >> k[i];
3  for(LL state = 0; state < 1LL << n; state++){          ----> A
4      a = 0, b = 0;
5      for(int i = 0; i < n; i++){                        ----> B
6          if(state >> i & 1) a += k[i];
7          else b += k[i];
8      }
9      ans = min(ans, max(a, b));                          ----> C

```

- state表示了一个被**压缩的状态** → 用long long 表示一个被分类的状态

A: 实际上是表示了 2^n 种状态 $1LL \ll n$ 等效于 2^n

B: 二进制位一共有n位, i 从零到 n 解 state二进制位

- 对于每个元素 `k[i]`, 根据 `state` 的二进制位判断该元素属于组 `a` 还是组 `b`

C: 刷新ans,这确保了 `ans` 始终存储的是所有分组方式中最小的最大和

例子解释, 当 $n = 3$ 时

每个 `state` 的二进制位表示每个元素属于哪一组。例如, 对于 $n = 3$ (有 3 个元素的情况):

- `state = 0` (000): 所有元素都在组 `b`
- `state = 1` (001): 第一个元素在组 `a`, 其余元素在组 `b`
- `state = 2` (010): 第二个元素在组 `a`, 其余在组 `b`
- `state = 3` (011): 第一个和第二个元素在组 `a`, 第三个在组 `b`
- ...
- `state = 7` (111): 所有元素都在组 `a`

12 哈希表

哈希表的实现可以由三种数据结构实现

set map unordered_map

- 在C++中，set 和 map 分别提供以下三种数据结构，其底层实现以及优劣如下表所示：

集合	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::set	红黑树	有序	否	否	$O(\log n)$	$O(\log n)$
std::multiset	红黑树	有序	是	否	$O(\log n)$	$O(\log n)$
std::unordered_set	哈希表	无序	否	否	$O(1)$	$O(1)$
- | 映射 | 底层实现 | 是否有序 | 数值是否可以重复 | 能否更改数值 | 查询效率 | 增删效率 |
|--------------------|------|-------|----------|---------|-------------|-------------|
| std::map | 红黑树 | key有序 | key不可重复 | key不可修改 | $O(\log n)$ | $O(\log n)$ |
| std::multimap | 红黑树 | key有序 | key可重复 | key不可修改 | $O(\log n)$ | $O(\log n)$ |
| std::unordered_map | 哈希表 | key无序 | key不可重复 | key不可修改 | $O(1)$ | $O(1)$ |

哈希表算法是一种以空间换时间的算法，可以在判断数据是否重复出现 or 数据去重方面有优异性能

例题：leetcode NO.1两数之和

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值** `target` 的那 **两个** 整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。

你可以按任意顺序返回答案。

哈希代码示例：< leetcode.1 >

```
1 class Solution {
2 public:
3     vector<int> twoSum(vector<int>& nums, int target) {
4         unordered_map<int, int> mp;
5         for (int i = 1; i <= nums.size(); i++) {
6             mp[nums[i - 1]] += i;
7             if (mp[target - nums[i]] != 0) {
8                 return {i, mp[target-nums[i]]-1};
9             }
10        }
11        return {};
12    }
13 };
14
```

13 前缀和与差分

13.1 一维数组的前缀和

- 对数列 $A[1, 2, 3, 4, 5]$, 求数列 B , 使 $B_i = \sum_{k=0}^i A_k$

思路: 因为 $B[0] = A[0]$ 且有递推式 $B_i = B_{i-1} + A_i$ 构造递推式

```
1  int main()
2  {
3      ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
4      vector<int> A = {1, 2, 3, 4, 5, 6};
5      vector<int> B(A.size());
6      B[0] = A[0];
7      for (size_t i = 1; i < A.size(); i++) {
8          B[i] = B[i - 1] + A[i];
9      }
10     for (auto &&i : B) cout << i << " ";
11     //输出: 1 3 6 10 15 21
12     return 0;
13 }
```

于传统代码相比, 将时间复杂度从 $O(n^2)$ 降低到 $O(n)$ 且空间复杂度仍然是 $O(n)$

13.2 二维数组的前缀和

对一个二维数组 A :

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

二维数组前缀和定义: 存在二维数组 S , 有:

$$S_{i,j} = \sum_{k_1=0}^i \sum_{k_2=0}^j A_{k_1,k_2}$$

类比一维的情形, $S_{i,j}$ 应该可以基于 $S_{i-1,j}$ 或 $S_{i,j-1}$ 计算, 从而避免重复计算前面若干项的和。但是, 如果直接将 $S_{i-1,j}$ 和 $S_{i,j-1}$ 相加, 再加上 $A_{i,j}$, 会导致重复计算 $S_{i-1,j-1}$ 这一重叠部分的前缀和, 所以还需要再将这部分减掉。这就是 [容斥原理](#)。由此得到如下递推关系:

$$S_{i,j} = A_{i,j} + S_{i-1,j} + S_{i,j-1} - S_{i-1,j-1}$$

简单代码实现:

```
1  int main()
2  {
3      ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
4      //在构建A的时候可以将A整体向右下移一个单位防止负数下标的出现
5      vector<vector<int>> A = {
6          {0, 0, 0, 0, 0},
7          {0, 1, 2, 3, 4},
8          {0, 5, 6, 7, 8},
```

```

9         {0, 9, 10, 11, 12},
10    };
11    vector<vector<int>> S(A.size(), vector<int>(A[0].size()));
12    S[1][1] = A[1][1];
13    for (size_t i = 1; i < A.size(); i++) {
14        for (size_t j = 1; j < A[0].size(); j++) {
15            if (i == j && i == 1) continue;
16            S[i][j] = A[i][j] + S[i - 1][j] + S[i][j - 1] - S[i - 1][j - 1];
17        }
18    }
19    for (auto &&i : S) {
20        for (auto &&j : i) {
21            cout << j << " ";
22        }
23        cout << endl;
24    }
25    /*输出:
26    0 0 0 0 0
27    0 1 3 6 10
28    0 6 14 24 36
29    0 15 33 54 78
30    */
31    return 0;
32 }

```

例: [洛谷 P1387 最大正方形](#)

最大正方形

题目描述

在一个 $n \times m$ 的只包含 0 和 1 的矩阵里找出一个不包含 0 的最大正方形，输出边长。

输入格式

输入文件第一行为两个整数 n, m ($1 \leq n, m \leq 100$)，接下来 n 行，每行 m 个数字，用空格隔开，0 或 1。

输出格式

一个整数，最大正方形的边长。

样例 #1

样例输入 #1

```

1  4 4
2  0 1 1 1
3  1 1 1 0
4  0 1 1 0
5  1 1 0 1

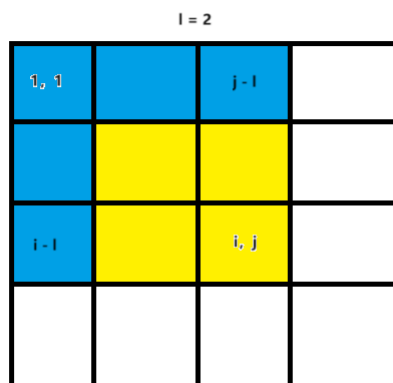
```

样例输出 #1

1 | 2

解决思路:

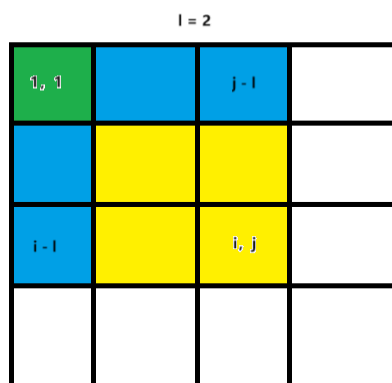
1. 先使用二维前缀和数组将输入数组 A 的 $A_{1,1} \rightarrow A_{i,j}$ 的和表示出来
2. 设立边长 l 从1开始找最小正方形, 其过程如下:
 1. 设立 i, j 作为假设正方形的右下角点, 找的正方形为以 l 为边长, 底点为 (i, j) 的一个正方形
 2. 显然 i, j 均从 l 开始, 到 m, n 结束
 3. $b[i][j] - b[i-l][j] - b[i][j-l] + b[i-l][j-l] == l * l$ 作为判断标准(why)
 - $b[i][j]$ 是从 $(1, 1) \rightarrow (i, j)$ 的前项和, 我们需要求 $(i-l, j-l) \rightarrow (i, j)$ 的和



■

1 |

图一



■

图二

- 不难发现: 我们想求的是黄色部分的前缀和并判断其是否等于 l^2
- 我们可以发现求黄色部分就是将 $b(i, j)$ 减去图一蓝色的部分, 可以减去两矩形的面积 $b(i-l, j), b(i, j-l)$, 但显然会多减去图二绿色部分的区域, 所以我们需要利用容斥定理加上 $b(i-l, j-l)$ 的面积
- 所以我们便得到了黄色部分的递推表达式:
$$locans = b_{i,j} - b_{i-l,j} - b_{i,j-l} + b_{i-l,j-l}$$
- 现在判断 $locans$ 与 l^2 的关系即可

- 完成一次查找之后就可以让 l 增加, 显然 $l \in [1, \min(n, m)]$

示例代码:

```
1  #include <algorithm>
2  #include <iostream>
3  using namespace std;
4  int a[103][103];
5  int b[103][103]; // 前缀和数组, 相当于上文的 sum[]
6
7  int main() {
8      int n, m;
9      cin >> n >> m;
10
11     for (int i = 1; i <= n; i++) {
12         for (int j = 1; j <= m; j++) {
13             cin >> a[i][j];
14             b[i][j] =
15                 b[i][j - 1] + b[i - 1][j] - b[i - 1][j - 1] + a[i][j]; // 求前缀和
16         }
17     }
18
19     int ans = 0;
20
21     int l = 1;
22     while (l <= min(n, m)) { // 判断条件
23         for (int i = l; i <= n; i++) {
24             for (int j = l; j <= m; j++) {
25                 if (b[i][j] - b[i - 1][j] - b[i][j - 1] + b[i - 1][j - 1] == 1 * 1)
26                 {
27                     ans = max(ans, l); // 在这里统计答案
28                 }
29             }
30             l++;
31         }
32
33         cout << ans << endl;
34         return 0;
35     }
```

Tip

上面进行的操作便是在前缀和中寻找某一区域的和的操作, 利用这个思想, 我们可以寻找任意大小的区域和, 可以从容斥定理入手, 进行区域和的计算

树上前缀和

• 一维数组树上前缀和

1. 在求解一维数组之前我们要进行一次前缀和操作
2. 对数组 A 的 $[l, r]$ 区间, 其区间和如下:

$$sum[l, r] = S[r] - S[l - 1]$$

时间复杂度: 构造前缀和数组 $O(n)$, 查询区间和 $O(1)$

- 二维数组树上前缀和

- 与一维数组树上前缀和一样, 二维树上前缀和也需要提前做好前缀和工作
- 快速求任意子矩阵的和

$$\text{sum}([x_1, y_1], [x_2, y_2]) = S[x_2][y_2] - S[x_1 - 1][y_2] - S[x_2][y_1 - 1] + S[x_1 - 1][y_1 - 1]$$

其中:

- $S[x_1][y_2]$: 从 $[1, 1]$ 到 $[x_1, y_1]$ 的矩形和;
- $S[x_2][y_2]$: 从 $[1, 1]$ 到 $[x_2, y_2]$ 的矩形和;
- $-S[x_1 - 1][y_2]$: 减去上方多余的矩形;
- $-S[x_2][y_1 - 1]$: 减去左侧多余的矩形;
- $+S[x_1 - 1][y_1 - 1]$: 加回左上角重复减去的部分。

💡 Tip

要注意的是在减去左边和上边的矩形的时候, 下标都为 $S[x_1 - 1][y_2]$ 和 $S[x_2][y_1 - 1]$

差分

- 指找数组范围内某一两项的差
- 定义: $\text{diff}[i] = a_i - a_{i-1}$ 特别的: $i = 1$ 时, $\text{diff}[1] = a_1$
- 差分和原数组的关系 **性质**:
 - 我们将差分数组做一次前项和得到的即为原数组
 - 显然: 我们对前缀和数组做一次差分得到的就是原数组

差分计算数组动态变化

- 对数组 A 的 $[l, r]$ 区间同时加 k 有:

```
1 diff[l] += k
2 diff[r+1] -= k
```

此时再进行一次前缀和, 即可完成原始数组的复原

```
1 for(int i = 1; i <= diff.size(); i++){
2     sum[i] = sum[i-1] + d[i];
3 }
```

14 BFS 广度优先搜索

- 在前面我们介绍过DFS深度优先搜索, 深搜的核心思想是一条路走到底, 直到得到符合的结果或者超出边界情况结束
- 而广度优先搜索则是从起始位置出发, 每一次向外增加一圈, 或执行完一大个操作之后再计数变量增加
 - 广搜一般适用于求最短路径, 求最少操作次数的这些操作, 因为广搜本身便是向外扩散式的一种搜索

广搜一般使用STL中的 `queue` 作为承接模板，通过其先进先出的特点实现广搜

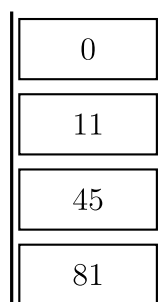
例题：[洛谷P1135 奇怪的电梯](#)

```
1 signed main()
2 {
3     int n, a, b;
4     cin >> n >> a >> b;
5     vector<int> to, path;
6     to = vector<int>(n + 1);
7     path = vector<int>(n + 1, -1);
8     for (size_t i = 1; i <= n; i++) {
9         cin >> to[i];
10    }
11    // 输入数据
12    queue<int> bfs; //建立一个bfs的queue队列
13    int ans = 0;
14    bfs.push(a); //将第一个元素存入队列中
15    path[a] = 0; //第一个路径初始
16    while (!bfs.empty()) { //如果队列非空，就说明仍然有可以进行下去的操作
17        int up = bfs.front() + to[bfs.front()]; //第一种可能，往上坐电梯
18        int down = bfs.front() - to[bfs.front()]; //第二种可能，往下坐电梯
19        if (up > 0 && up <= n && path[up] == -1) { //如果往上做的电梯能到达(即存
20            path[up] = path[bfs.front()] + 1; // 这个楼层的标识数 = 过来的楼层的
21            bfs.push(up); //把这个楼层加入到队列里，表示接下来会对这个楼层操作
22        }
23        if (down > 0 && down <= n && path[down] == -1) { //同上
24            path[down] = path[bfs.front()] + 1;
25            bfs.push(down);
26        }
27        bfs.pop(); //原始楼层操作完毕，弹出队列
28    }
29    cout << path[b]; //输出目标楼层的情况
30    return 0;
31 }
```

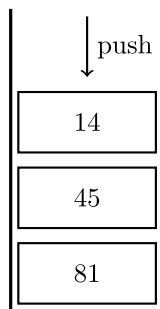
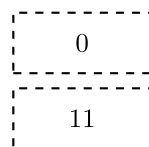
显然，BFS对这种求最短是一个不错的解法，但有的时候还得考虑DP或其他时间复杂度更低的方法

15 单调栈

- 何为单调栈？顾名思义，单调栈即满足单调性的栈结构。与单调队列相比，其只在一端进行进出。
- 将一个元素插入单调栈时，为了维护栈的单调性，需要在保证将该元素插入到栈顶后整个栈满足单调性的前提下弹出最少的元素。



如上：一个 {0,11,45,81} 的单调栈，如果要插入元素 14



必须将{0,11} 弹出栈，再放入 元素 14

```
1 insert x;
2 while (!sta.empty() && sta.top() <= x) {
3     sta.pop();
4 }
5 sta.push(x);
```

💡 Tip

有的人会问，那被弹出去的元素呢？其实，在大部分单调栈问题中，我们更关注在一次弹出后的栈的状态，即在放入当前元素前栈的各个属性，比如：

- 栈顶 → 能告诉我们比这个元素大(小)的元素是什么
- 栈的大小 → 能告诉我们在这个元素前有几个比当前元素大(小)的元素

例题：

[洛谷P5788 单调栈\(模板\)](#)

```
1 signed main()
2 {
3     //ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
4     int t = 1;
5     cin >> t;
6     vint n(t);
7     for (size_t i = 0; i < t; i++) {
8         cin >> n[i];
9     }
10    stack<int> sta;
11    vint result(t, 0);
12    for (size_t i = t - 1; i >= 0; i--) {
13        while (!sta.empty() && n[sta.top()] <= n[i]) {
14            sta.pop();
15        }
16        result[i] = sta.empty() ? 0 : sta.top() + 1;
17        sta.push(i);
18    }
19    for (auto &&i : result) {
```

```

20     cout << i << " ";
21 }
22 return 0;
23 }

```

思路解释:

- 维护一个单调存放数字下标的单调栈，这个单调栈入栈的规则根据数字的具体大小决定
- 当某个元素入栈时，说明至少这个元素会比当前的栈顶下标所代表的元素要大，所以我们便把栈顶弹出，直到找到某个比当前元素大的数
- 而找到的这个正好就是我们要找的刚好大于这个数，我们就被栈顶放入 result 数组中即可
- 那为什么栈顶就是我们要找的数字呢？
 - 如 2 6 5 7 5
 - 进行比较的是与元素的大小，但栈存放的是下标 → 所以使用 `n[sta.top()] <=` `n[i]` 进行下标栈和元素大小栈的转换
 - 我们最先放进去的是 5，此时栈的状态是 { 5 }，→ 下标栈的状态 { 5 }
 - 接着我们要放入 7，不难发现 $7 > 5$ ，弹出 5 放入 7
 - `sta : { 4 }` —— `num { 7 }`
 - 再看 5， $5 < 7$ ，不需要弹出，直接放入，此时比 5 大的数就是 7，在 result 数组下放入 `sta.top()` 即可
 - `sta : { 3 4 }` —— `num { 5 7 }`
 - 看 6，6 比 5 大，比 7 小，将 5 弹出，放入 7 所代表的下标作为答案
 - 以此类推即可

例题2:

[\[USACO06NOV\] Bad Hair Day S](#)

```

1  signed main()
2  {
3      //ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
4      int t = 1, ans = 0;
5      cin >> t;
6      stack<int> cowh;
7      for (size_t i = 1; i <= t; i++) {
8          int h;
9          cin >> h;
10         while (!cowh.empty() && cowh.top() <= h) {
11             cowh.pop();
12         }
13         ans += cowh.size();
14         cowh.push(h);
15     }
16     cout << ans;
17     return 0;
18 }

```

思路解释

- 与上一题相似，但这次我们要找的是前面有多少个比放入的数字大的数量

- 还是维护一个栈，如果当前放入的数字比栈顶大就弹出栈顶
- 我们 ans 加的是还放入这个数的栈的大小，代表在这之前比这个数字大的数字有多少个
- 我们要求的是 **这个数能看见多少个**，求所有数能看见的总和 同样可以标识为 **这个数能被多少个看见，每个数能被看见的总和**
- 那当我们弹出比这个数小的数字之后，剩下的数就一定都能看见这个数
- 将这些数量加到 ans 内即可

16 约瑟夫问题

人们站在一个等待被处决的圈子里。计数从圆圈中的指定点开始，并沿指定方向围绕圆圈进行。在跳过指定数量的人之后，处刑下一个人。对剩下的人重复该过程，从下一个人开始，朝同一方向跳过相同数量的人，直到只剩下一个人，并被释放。

现在我想知道幸存者的编号(编号从0开始)的多少

Tip

考虑在有 $n - 1$ 个人，处决 k 的情况下最后一个人的编号是 $f(n - 1, k)$ ，那么对 n 个人而言，有递推式 $f(n, k) = (f(n - 1, k) + k) \bmod n$

递推式求解抽象化：考虑对 $f(n - 1, k)$ ，扩展到 n 人时，我们只需要将第 k 个人处决，问题便又回到了对 $n - 1$ 人中找 $f(n - 1, k)$ 的问题上，但我们删除编号 k 个数时候，所有数的编号都会进行变化映射，即新的环的起点发生了变化，即集体向后移动 k 位且保证在 n 范围内，所以需要取模与 n

即： $f(n, k) = (f(n - 1, k) + k) \bmod n$

```
1 int josephus(int n, int k)
2 {
3     int s = 0;
4     for (int i = 2; i <= n; i++) s = (s + k) % i;
5     return s + 1;
6 }
```

上述算法时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ ，但对 $k = 2$ 的情况我们可以有特殊解法使时间复杂度降低到 $O(\log n)$ ：引用自 [Wikipedia](#)

答案的最漂亮的形式，与 (n) 的二进制表示有关：把 n 的第一位移动到最后，便得到 $f(n)$ 。如果 n 的二进制表示为 $n = b_0b_1b_2b_3 \dots b_m$ 则 $f(n) = b_1b_2b_3 \dots b_mb_0$ 。这可以通过把 n 表示为 $2^m + l$ 来证明。

```
1 int yuesefu(int n)
2 {
3     // 找到最高位的值，即n的最高位所代表的2的幂
4     int highestBit = 1;
5     while (highestBit <= n) {
6         highestBit <<= 1; // 将最高位左移
7     }
8     // 移动后的结果
9     return (n - (highestBit >> 1)) << 1 + 1;
10    // return (n - highestBit / 2) * 2 + 1
11 }
```

17 最长回文子序列

给定一个字符串 s ，求 s 的最长回文子序列

17.1 模拟暴力法

- 通过一个二维数组遍历原字符串的所有子串
- 逐一判断该子串是否为回文串
- 更新最长子串

```
1  #pragma GCC optimize(3)
2  #include <bits/stdc++.h>
3  // #define int LL
4  #define endl '\n'
5  #define size_t int
6  #define all(v) v.begin(), v.end()
7  using namespace std;
8  typedef long long LL;
9  typedef vector<int> vint;
10 typedef vector<vint> vvint;
11 typedef vector<string> vstr;
12 typedef pair<int, int> pii;
13 typedef vector<pii> vpii;
14
15 string huiwen(const string &s, int back, int front)
16 {
17     int idx_b = back;
18     int idx_f = front;
19     while (idx_b > idx_f) {
20         if (s[idx_f] != s[idx_b]) {
21             return s.substr(0, 1);
22         }
23         idx_f++;
24         idx_b--;
25     }
26     string res;
27     for (size_t i = front; i <= back; i++) {
28         res += s[i];
29     }
30     return res;
31 }
32
33 signed main()
34 {
35     // ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
36     string s;
37     cin >> s;
38     string ans = "";
39     if (s.size() <= 1) {
40         cout << s;
41     }
42     for (size_t i = 0; i < s.size(); i++) {
43         for (size_t j = i + 1; j < s.size(); j++) {
44             string temp = huiwen(s, j, i);
```

```

45         ans = (temp.size() > ans.size() ? temp : ans);
46     }
47 }
48 cout << ans;
49 return 0;
50 }

```

这个算法的复杂度来到了惊人的 $O(n^3)$,令人汗颜,数据量在 10^3 就能直接让OJ爆TLE

17.2 中心扩展算法

中心算法的三个核心步骤

- **1.选择中心**: 对于一个长度为 n 的字符串, 其拥有 $2n - 1$ 个中心结点 (即长度为奇数的回文子串结点有 n 个, 长度为偶数的回文子串的结点有 $n - 1$ 个)
- **2.扩展回文**: 对于每个选定的中心, 尝试向两边扩展, 直到无法形成回文为止
- **3.更新结果**: 在每次成功扩展后, 计算当前回文的长度, 并与已知的最大回文长度进行比较。如果更长, 则更新最大回文及其起始位置

```

1  // #pragma GCC optimize(3)
2  #include <bits/stdc++.h>
3  // #define int LL
4  #define endl '\n'
5  #define size_t int
6  #define all(v) v.begin(), v.end()
7  using namespace std;
8  typedef long long LL;
9  typedef vector<int> vint;
10 typedef vector<vint> vvint;
11 typedef vector<string> vstr;
12 typedef pair<int, int> pii;
13 typedef vector<pii> vpii;
14
15 int expandAroundCenter(const string &s, int left, int right)
16 {
17     while (left >= 0 && right < s.size() && s[left] == s[right]) {
18         left--;
19         right++;
20     }
21     return right - left - 1;
22 }
23
24 string longestPalindrome(const string &s)
25 {
26     if (s.empty()) return "";
27     int start = 0;
28     int end = 0;
29     for (size_t i = 0; i < s.size(); i++) {
30         int len1 = expandAroundCenter(s, i, i);
31         int len2 = expandAroundCenter(s, i, i + 1);
32         int len = max(len1, len2);
33         if (len > end - start) {
34             end = i + len / 2;
35             start = i - (len - 1) / 2;

```

```

36     }
37 }
38 return s.substr(start, end - start + 1);
39 }
40
41 signed main()
42 {
43     //ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
44     string s;
45     cin >> s;
46     cout << longestPalindrome(s) << endl;
47     return 0;
48 }

```

此时时间复杂度已经降至 $O(n^2)$ ，空间复杂度降低至 $O(1)$ ，已经绝杀大部分算法

17.3 Manacher's Algorithm

很抱歉没能让manacher大人尽兴 ——by 中心扩展算法

- **Manacher's Algorithm(马拉车算法)**是一种高效解决最长回文子串的算法，其提供一个 $O(n)$ 复杂度下的方法来实现对回文子序列的查找
 - 核心思想
 - **回文串的对称性**：假设已经计算出了以某个位置为中心的回文串的半径（即该回文串两边的最大扩展长度），那么与其对称的部分的回文半径也可以直接推算出来
 - **预处理**：将原字符串进行预处理，以避免处理奇数长度和偶数长度回文的不同情况。通常的做法是在每个字符之间插入一个特殊字符（如 #），并在字符串的开头和结尾添加不同的特殊字符（如 ^ 和 \$）。这样处理后的字符串长度一定是奇数，且不会出现冲突。
 - **动态维护最远回文边界**：在扩展时，维护一个“右边界” **R** 和一个“中心” **C**，**C** 代表回文串的中心位置，**R** 代表回文串的最右边界。当当前处理的位置 **i** 位于 **R** 的范围内时，可以利用已知的对称性质来减少计算。

```

1  // #pragma GCC optimize(3)
2  #include <bits/stdc++.h>
3  // #define int LL
4  #define endl '\n'
5  #define size_t int
6  #define all(v) v.begin(), v.end()
7  using namespace std;
8  typedef long long LL;
9  typedef vector<int> vint;
10 typedef vector<vint> vvint;
11 typedef vector<string> vstr;
12 typedef pair<int, int> pii;
13 typedef vector<pii> vpii;
14
15 // 预处理
16 string preprocess(const string &s)
17 {
18     string t = "^";
19     for (char c : s) {
20         t += "#" + string(1, c);
21     }

```



```

22     t += "#$";
23     return t;
24 }
25
26 string longestPalindrome(const string &s)
27 {
28     string T = preprocess(s);
29     int n = T.size();
30     vector<int> P(n, 0); //P[i] 记录以 t[i] 为中心的回文半径
31     int C = 0, R = 0;    //C 是回文中心, R 是回文串的最右边界
32     for (size_t i = 1; i < n - 1; i++) {
33         // 确定对称位置
34         int Mirror = 2 * C - i;
35         if (i < R) {
36             P[i] = min(P[Mirror], R - i);
37         }
38         // 尝试扩展边界
39         while (T[i + 1 + P[i]] == T[i - 1 - P[i]]) {
40             P[i]++;
41         }
42         // 如果当前回文串扩展超过了 R, 更新中心和右边界
43         if (i + P[i] > R) {
44             C = i;
45             R = i + P[i];
46         }
47     }
48     // 找到最长的回文子串
49     int maxLen = 0;
50     int Centerindex = 0;
51     for (size_t i = 0; i < n - 1; i++) {
52         if (P[i] > maxLen) {
53             maxLen = P[i];
54             Centerindex = i;
55         }
56     }
57     // 构造回文串
58     int start = (Centerindex - maxLen) / 2;
59     return s.substr(start, maxLen);
60 }
61
62 signed main()
63 {
64     //ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
65     string s;
66     cin >> s;
67     cout << longestPalindrome(s) << endl;
68     return 0;
69 }

```

注释

可以这么表示

```
1  int arr[2][3] =  
2  {  
3  {1,1,4},  
4  {5,1,4}  
5  };
```

-
1. 如果函数内置了比较器(sort,优先队列),那大部分默认使用 `less<int>` [↩](#)
 2. 如果没有特殊说明, 本条目下所有 `str` 均表示字符串名 [↩](#)
 3. 没有特殊说明, 本条目下所有 `vec` 均表示容器名 [↩](#)
 4. 如果无特殊说明, 本条目下所有 `dp` 均表示容器名 [↩](#)
 5. `.insert()` 成员函数对vector容器也适用,但插入元素可能倒置vector容器重新分配内存导致STL [↩](#)